

DESIGN AND IMPLEMENTATION OF A RAG PIPELINE EVALUATION AND OBSERVABILITY SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by:

Revanth Chary Roll No: [Roll Number]

Under the Guidance of:

[Guide Name] [Designation], Department of Computer Science and Engineering

Department of Computer Science and Engineering

[College Name]

[University Name]

[City], [State] — [PIN Code]

[Academic Year: 2024–2025]

Bonafide Certificate

This is to certify that the project work entitled “**Design and Implementation of a RAG Pipeline Evaluation and Observability System**” is a bonafide record of the work done by **Revant Chary**, bearing Roll No. **[Roll Number]**, student of **B.Tech in Computer Science and Engineering**, [College Name], affiliated to [University Name], in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering during the academic year [Year]–[Year].

This project work has been carried out under our guidance and supervision. To the best of our knowledge and belief, the work presented in this report has not been submitted elsewhere for the award of any other degree, diploma, or similar award.

Internal Guide:

[Guide Name] [Designation] Department of Computer Science and Engineering [College Name]

Head of Department:

[HOD Name] Professor and Head Department of Computer Science and Engineering [College Name]

External Examiner:

[Name and Designation] [Organization / Institute]

Examination Date: _____

Student Declaration

I, **Revanth Chary**, student of B.Tech (Computer Science and Engineering), Roll No. [Roll Number], [College Name], affiliated to [University Name], hereby solemnly declare that the project entitled “**Design and Implementation of a RAG Pipeline Evaluation and Observability System**” submitted to the Department of Computer Science and Engineering in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology is an original piece of work carried out by me independently under the guidance of [**Guide Name**], [Designation], Department of CSE.

I further declare that:

1. The work presented in this project report is entirely my own and has not been submitted to any other institution or university for the award of any degree, diploma, or any other academic distinction.
2. Wherever I have used information, ideas, or data from other sources, I have duly acknowledged the same at the appropriate place in the report by citing proper references.
3. I have not violated any intellectual property rights of any individual or organisation while preparing this report or while implementing the project.
4. The source code developed as part of this project has not been plagiarised from any publicly available repository without proper attribution and modification.
5. The results and conclusions presented in this report are based on actual experiments carried out by me.

Place: [City]

Date: [Date]

Signature of Student:

Revanth Chary Roll No: [Roll Number] B.Tech — CSE [College Name]

Acknowledgement

There are many people without whose help and support this project would not have been possible, and I would like to take this opportunity to express my heartfelt gratitude to all of them.

First and foremost, I am deeply grateful to my project guide, **[Guide Name]**, [Designation], Department of Computer Science and Engineering, [College Name], for the immense support, patient guidance, and timely feedback offered throughout the course of this project. The discussions we had during the weekly review meetings helped me think more clearly about several design decisions, especially around the evaluation architecture and the observability layer. Their willingness to engage with both the technical depth and the broader academic framing of this work made a real difference.

I sincerely thank **[HOD Name]**, Professor and Head, Department of Computer Science and Engineering, [College Name], for providing the necessary infrastructure, laboratory facilities, and a supportive academic environment that made this work possible. The department's emphasis on practical, implementable projects encouraged me to pursue something that goes beyond a basic demonstration.

I am grateful to the **Principal and Management** of [College Name] for their continuous encouragement of student-driven research and development activities, and for the computing resources that were made available to students of this department.

This project uses several outstanding open-source technologies, and I would like to acknowledge the communities behind **FastAPI**, **Next.js**, **PostgreSQL**, **pgvector**, **OpenTelemetry**, **Prometheus**, and **Grafana** — all of which are freely available and collectively made this system possible without any licensing cost. I am also grateful to **OpenAI** for providing the embedding and language model APIs that form the core of the RAG pipeline.

I owe a special thanks to my classmates and friends who offered feedback on the user interface, tested early versions of the application, and pointed out bugs I had long stopped noticing. Fresh eyes matter more than one realises during a project of this scale.

Finally, and most importantly, I thank my **family** for their unconditional support, for tolerating the late nights, and for believing in this work even during the periods when I was less certain about it myself.

Abstract

Retrieval-Augmented Generation, or RAG, has over the past few years become one of the most widely adopted architectural patterns for building intelligent document question-answering systems. By combining vector-based semantic search with the generative capabilities of large language models, RAG systems allow users to ask natural language questions about their own private

documents and receive answers that are grounded in retrieved evidence rather than generated from memory. This approach addresses a fundamental limitation of standalone language models — their inability to access private, proprietary, or up-to-date information without expensive fine-tuning or retraining.

However, while building a working RAG chatbot has become relatively straightforward with the availability of modern tools and APIs, operating and maintaining one reliably over time is a significantly harder problem that the field has not yet solved in a satisfactory way. Most RAG deployments lack the infrastructure to continuously measure retrieval quality, detect regressions when the system changes, or explain exactly why a particular answer was slow or incorrect. This gap — between building a demo and building a production-grade system — is what this project is designed to address.

This report describes the design and implementation of a **RAG Pipeline Evaluation and Observability System**: a full-stack, self-contained, deployable platform that integrates document-grounded chat, offline evaluation, a merge-blocking CI regression gate, distributed pipeline tracing, and a query-log explorer within a single codebase backed by a shared PostgreSQL database.

The system supports four interchangeable retrieval strategies — vector similarity search, hybrid BM25-plus-vector search with reciprocal rank fusion, LLM-based reranking, and multi-query retrieval — each of which can be benchmarked against a reproducible evaluation dataset. The evaluation harness computes standard information retrieval metrics including Hit@1, Hit@3, Hit@5, Hit@8, and Mean Reciprocal Rank (MRR), persists results in PostgreSQL with stable identifiers, and provides a frontend interface for listing, drilling into, and comparing runs side by side.

The observability layer instruments every stage of the RAG pipeline with OpenTelemetry spans, exports latency percentiles (p50, p95, p99) as Prometheus histograms, and provides a query-log explorer that correlates live production traffic with offline evaluation failures through a shared query identifier. A worked regression case study demonstrates the CI gate catching a real degradation: ingesting four broad overview documents caused MRR to drop from 0.840 to 0.812, and the gate blocked the merge — while a Hit@5-only gate would have missed the regression entirely.

The system is built on Next.js 15 and React 19 for the frontend, FastAPI (Python 3.11) for the backend, and PostgreSQL with pgvector for vector storage and retrieval. It is fully containerised with Docker Compose and includes cloud deployment guides.

Keywords: Retrieval-Augmented Generation, Evaluation Harness, Observability, OpenTelemetry, Semantic Search, pgvector, CI Regression Gate, FastAPI, Next.js, Information Retrieval

Table of Contents

S.No.	Chapter / Section	Page No.
	Cover Page	i
	Bonafide Certificate	ii
	Student Declaration	iii
	Acknowledgement	iv
	Abstract	v
	Table of Contents	vi
	List of Figures	viii
	List of Tables	ix
	List of Abbreviations	x
1	Introduction	1
1.1	Introduction	1
1.2	Background	3
1.3	Problem Statement	7
1.4	Objectives of the Project	9
1.5	Scope of the Project	10
1.6	Existing System and Its Drawbacks	11
1.7	Proposed System	13
1.8	Advantages of the Proposed System	15
2	Literature Survey	17
2.1	Review of Related Work	17
2.2	Study of Existing Technologies	23
2.3	Comparison of Previous Systems	29
3	System Analysis and Requirements	31
3.1	Requirement Analysis	31
3.2	Functional Requirements	32
3.3	Non-Functional Requirements	35
3.4	Feasibility Study	37
4	System Design	42
4.1	System Architecture	42
4.2	UML Diagrams	46
4.3	Database Design	54

S.No.	Chapter / Section	Page No.
4.4	Data Flow Diagrams	57
5	System Implementation	60
5.1	Hardware Requirements	60
5.2	Software Requirements	61
5.3	Development Environment Setup	62
5.4	Modules Description	64
5.5	Algorithms	74
5.6	Database Tables	78
5.7	Important Source Code	82
5.8	Application Screenshots	88
6	Testing and Results	90
6.1	Test Plan	90
6.2	Test Cases	91
6.3	Unit Testing	97
6.4	Integration Testing	98
6.5	System Testing	100
6.6	Results and Analysis	101
7	Conclusion and Future Scope	105
7.1	Conclusion	105
7.2	Limitations	107
7.3	Future Enhancements	108
	References / Bibliography	111
	Appendix A — Source Code (Key Modules)	115
	Appendix B — SQL Scripts	121
	Appendix C — User Manual	124
	Appendix D — Output Screenshots	128

List of Figures

Fig. No.	Figure Title	Page No.
4.1	Three-Tier System Architecture	43
4.2	RAG Pipeline — Request Processing Flow	45
4.3	OpenTelemetry Trace Pipeline Diagram	46
4.4	Use Case Diagram — System Overview	47
4.5	Class Diagram — Backend RAG Modules	48
4.6	Sequence Diagram — User Query Flow	49
4.7	Sequence Diagram — Document Ingestion	51
4.8	Sequence Diagram — Evaluation Run	52
4.9	Activity Diagram — CI Regression Gate	53
4.10	Activity Diagram — Chat with Citations	54
4.11	Entity Relationship (ER) Diagram	55
4.12	Data Flow Diagram — Level 0 (Context Diagram)	57
4.13	Data Flow Diagram — Level 1	58
4.14	Data Flow Diagram — Level 2 (Query Subsystem)	59
5.1	Module Interaction and Dependency Diagram	64
5.2	Adaptive Chunking Decision Tree	74
5.3	Reciprocal Rank Fusion (RRF) Process	76
5.4	Evaluation Harness Algorithm Flow	77
5.5	Chat Interface with Inline Citations	88
5.6	Evaluation Run List View	88
5.7	Evaluation Run Comparison View	89
5.8	Query Log Explorer	89
5.9	System Metrics Dashboard (Grafana)	90
6.1	Retrieval Strategy Benchmark Bar Chart	102
6.2	Latency Distribution per Retrieval Strategy	103
6.3	CI Regression Gate — Terminal Output (Exit 1)	104
6.4	MRR and Hit@1 Drop in Regression Case Study	104

List of Tables

Table No.	Table Title	Page No.
2.1	Summary of Key Research Papers Reviewed	22
2.2	Comparison of Existing RAG Systems and Tools	30
3.1	Functional Requirements	33
3.2	Non-Functional Requirements	36
3.3	Technical Feasibility Assessment	38
3.4	Economic Cost Estimate	39
5.1	Hardware Requirements	60
5.2	Software Stack — Backend	61
5.3	Software Stack — Frontend	62
5.4	Database Table — documents	78
5.5	Database Table — chunks	79
5.6	Database Table — queries	80
5.7	Database Table — eval_runs	81
5.8	Database Table — eval_case_results	82
6.1	Unit Test Cases — Backend	91
6.2	Unit Test Cases — Evaluation Module	93
6.3	Integration Test Cases — API Endpoints	94
6.4	System Test Cases — End-to-End Scenarios	96
6.5	Retrieval Strategy Benchmark Results	102
6.6	Regression Case Study — Metric Comparison	103

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BM25	Best Match 25 (probabilistic ranking function)
CI/CD	Continuous Integration / Continuous Deployment
CNCF	Cloud Native Computing Foundation
CSV	Comma-Separated Values
CSE	Computer Science and Engineering
DFD	Data Flow Diagram
DOCX	Microsoft Word Open XML Document Format
ER	Entity Relationship
FAQ	Frequently Asked Questions
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World (graph index)
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
JSON	JavaScript Object Notation
LLM	Large Language Model
MRR	Mean Reciprocal Rank
NLP	Natural Language Processing
OpenAI	Open Artificial Intelligence (company name)
ORM	Object Relational Mapper
OTLP	OpenTelemetry Protocol
OTel	OpenTelemetry
PDF	Portable Document Format
p50/p95/p99	50th, 95th, 99th Percentile Latency
PR	Pull Request
QA	Question Answering
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
REST	Representational State Transfer

Abbreviation	Full Form
RRF	Reciprocal Rank Fusion
SLO	Service Level Objective
SQL	Structured Query Language
SRE	Site Reliability Engineering
SSE	Server-Sent Events
TXT	Plain Text File Format
UML	Unified Modeling Language
UUID	Universally Unique Identifier
WSGI	Web Server Gateway Interface

CHAPTER 1

INTRODUCTION

1.1 Introduction

We are living through a period of rapid and genuine transformation in the way computers understand and respond to human language. Over the past few years, large language models have moved from being research curiosities confined to academic papers to being practical tools that developers can call with a few lines of code. The underlying capability — the ability to take in a block of text, understand its meaning, and produce fluent, contextually appropriate responses — has improved to a point where these models are genuinely useful for a wide range of real-world tasks.

One of the most natural and immediately valuable applications of this capability is document question-answering: giving a user the ability to ask questions about a collection of documents in plain English and receive accurate, cited answers. Businesses have enormous quantities of internal documentation, research reports, product manuals, legal agreements, and technical specifications that are difficult to search through effectively. Traditional keyword search returns documents but does not synthesise answers. A well-built AI-powered question-answering system can change this dramatically — allowing a user to ask “What is our refund policy for international orders?” or “What does the contract say about data retention?” and receive a direct, cited answer in seconds.

Retrieval-Augmented Generation, commonly referred to as RAG, has emerged as the dominant architectural approach for building such systems. The core idea is elegant in its simplicity: rather than relying entirely on what a language model has memorised during its training, the system retrieves relevant document chunks from a knowledge base at query time and provides them to the model as context. The model then generates an answer grounded in that retrieved evidence, citing the sources it used. This approach gives the model access to information it was never trained on, keeps the answers factually anchored to actual documents, and provides traceability — the user can see which parts of which documents the answer came from.

The practical appeal of RAG is significant. Building a fine-tuned model that knows all of your company’s internal documents requires enormous compute resources, specialised expertise, and months of effort — and the model must be retrained every time the documents are updated. A RAG system, by contrast, can be updated simply by ingesting new documents into the vector database. The knowledge base stays current; the language model stays the same. This makes RAG the practical choice for the vast majority of real-world document QA applications.

Despite its widespread adoption, however, there is a problem that becomes apparent the moment you try to operate a RAG system beyond its initial demo state: it is very hard to know whether it is working well. A RAG answer looks plausible whether the retrieved chunks were exactly right or subtly wrong. A pipeline change — a different chunking size, a new batch of documents, a modified embedding model — might improve answers for some questions and degrade them for others,

and there is typically no automatic way to detect this. Teams discover problems through user complaints, which arrive weeks after the regression was introduced and provide no indication of which change caused it.

This project was built to address that gap directly. The system described in this report does not merely implement a RAG chat interface — it wraps that interface in the kind of engineering infrastructure that is standard practice for conventional software but almost entirely absent from current RAG deployments. Specifically, it adds a persisted evaluation harness that runs the same dataset against the system after every change, a CI gate that blocks merges when retrieval quality drops, distributed tracing that attributes latency to specific pipeline stages, and a query-log explorer that lets a developer inspect any query — from CI or from a real user — in full detail. Every component shares the same PostgreSQL database, so the connection between a failing CI evaluation case and the production query that reveals why it failed is just a link to click.

This report is organised as follows. Chapter 1 provides background on the technology, defines the problem being solved, and describes the proposed system. Chapter 2 reviews related literature and existing tools. Chapter 3 covers system analysis and requirements. Chapter 4 describes the system design including architecture and UML models. Chapter 5 details the implementation. Chapter 6 presents testing results. Chapter 7 concludes the report and discusses future directions.

1.2 Background

1.2.1 The Evolution of Language Models

The story of large language models as we know them today begins with the Transformer architecture, introduced by Vaswani et al. in 2017. The key innovation of the Transformer was the self-attention mechanism, which allows the model to weigh the relevance of every part of an input sequence when generating each part of an output sequence. Unlike recurrent neural networks, which process text sequentially and struggle to capture long-range dependencies, Transformers can process entire sequences in parallel and attend to distant context with equal ease.

The next major step was the development of pre-trained language models — large Transformers trained on vast corpora of text to develop a general understanding of language before being fine-tuned on specific tasks. BERT (Devlin et al., 2018) demonstrated that a single pre-trained model could be fine-tuned with relatively little task-specific data to achieve state-of-the-art performance on a wide range of NLP benchmarks. The GPT series (OpenAI, 2018–2023) took a different approach — training large autoregressive models capable of generating coherent long-form text — and culminated in GPT-3 and GPT-4, models large enough that they can perform many tasks simply by being given instructions in natural language, without any fine-tuning.

These models — GPT-4, Claude, Gemini, and their contemporaries — are genuinely capable of understanding complex questions and generating high-quality, fluent answers. However, they have a fundamental limitation that makes them unsuitable for document question-answering on their own: their knowledge is frozen at the time of training. They do not know about documents created after their training cutoff, and they have no access to private organisational documents. Trying to

make them answer questions about specific documents by embedding the documents in the prompt is possible but limited — the context window has a finite size, and processing large document collections this way is prohibitively expensive.

1.2.2 Retrieval-Augmented Generation

The RAG paradigm was formalised in a 2020 paper by Lewis et al. at Facebook AI Research, though similar ideas had appeared in earlier work on knowledge-grounded dialogue systems. The key insight is that retrieval and generation can be combined into a single end-to-end trainable system: a retrieval component selects relevant documents, and a generation component conditions its output on both the query and the retrieved documents.

In the most common practical implementation — often called “naive RAG” — the process works in two phases. During the indexing phase, each document is split into smaller segments called chunks, and each chunk is converted into a dense vector embedding using an embedding model. These embeddings are stored in a vector database alongside the original text. During the query phase, the user’s question is embedded into the same vector space, the database returns the most similar chunks (by cosine similarity), and the language model generates an answer conditioned on the question and the retrieved chunks.

The quality of a RAG system depends critically on the retrieval step. If the right chunks are not retrieved, the language model cannot generate a correct answer regardless of its capabilities. This makes retrieval quality the central engineering concern in RAG systems — and the central focus of the evaluation harness built in this project.

1.2.3 Vector Embeddings and Semantic Search

At the heart of RAG retrieval is the concept of vector embeddings — numerical representations of text that capture semantic meaning. An embedding model (such as OpenAI’s text-embedding-3-small used in this project) converts a piece of text into a vector of floating-point numbers, typically with hundreds or thousands of dimensions. Texts that are semantically similar end up with embeddings that are close together in this high-dimensional space, as measured by cosine similarity or inner product.

This is fundamentally different from traditional keyword-based search. Keyword search (BM25, for instance) scores documents by counting how many of the query’s words appear in the document and how frequently. It works well when queries use the exact same vocabulary as the documents, but fails when the user asks a question using different words to express the same concept. Semantic search using embeddings handles this gracefully: a query about “how to cancel a subscription” and a document section about “membership termination procedures” will have similar embeddings even though they share no keywords, because the embedding model has learned that these concepts are related.

The practical implementation of semantic search at scale requires efficient approximate nearest-neighbour (ANN) algorithms. Searching through millions of embeddings by brute-force comparison would be too slow. Indexing structures like HNSW (Hierarchical Navigable Small World

graphs) provide fast approximate search with controllable accuracy-speed trade-offs. The pgvector extension for PostgreSQL implements both IVFFlat and HNSW indexes, enabling semantic search directly within a relational database — which is what this project uses.

1.2.4 Hybrid Retrieval: Combining Sparse and Dense Search

Pure vector similarity search is not always the best retrieval strategy. Research has consistently shown that combining dense (vector) retrieval with sparse (keyword) retrieval often outperforms either alone, particularly for queries that contain specific technical terms, proper nouns, or rare identifiers that are important to match exactly.

BM25 (Best Match 25) is the dominant sparse retrieval algorithm, derived from the probabilistic relevance model developed by Robertson and Spärck Jones. It ranks documents by a weighted combination of term frequency and inverse document frequency, with saturation functions that prevent any single highly-frequent term from dominating the score. BM25 is fast, interpretable, and very effective at finding documents that contain the exact words the user used.

Combining BM25 and vector search results requires a method for merging two different scoring systems whose scales are not comparable. Reciprocal Rank Fusion (RRF), introduced by Cormack et al. in 2009, solves this elegantly: instead of trying to normalise scores, it works only with ranks. Each document's RRF score is the sum of reciprocal rank values across all retrieval systems, with a smoothing constant to prevent very high scores for top-ranked documents. This fusion approach is simple, robust, and consistently competitive with more complex fusion methods.

1.2.5 The Reranking Approach

A two-stage retrieval pipeline separates retrieval into a fast first stage that casts a wide net and a slower second stage that scores candidates more carefully. The first stage (typically BM25 or vector search) retrieves a large set of candidate documents. The second stage (a reranker) scores each candidate using a more expensive model that can consider the full query-document interaction.

Cross-encoder rerankers, which jointly encode the query and each candidate document through a transformer, are significantly more accurate than bi-encoder retrievers but cannot be applied to an entire corpus (they scale quadratically with corpus size). Used as a reranker over a candidate set of, say, 20 documents, they are computationally feasible and can substantially improve precision.

In this project, the reranking strategy uses the language model itself as the reranker: it prompts the LLM to assign a relevance score to each retrieved candidate, then returns the top-k by LLM score. This is slower and more expensive than the initial retrieval, but it can correct cases where a semantically similar chunk is retrieved but does not actually answer the question.

1.2.6 Evaluation Metrics in Information Retrieval

Measuring the quality of a retrieval system requires metrics that capture how well the system ranks relevant documents. This project uses two families of metrics.

Hit@k (also written Recall@k or HR@k) measures whether the relevant document appears anywhere in the top-k retrieved results. Hit@1 is 1 if the correct document is the first result, 0 otherwise. Hit@5 is 1 if the correct document appears anywhere in the top 5 results. Averaged across

all queries in a dataset, these metrics give the fraction of queries for which the system retrieved the correct document within the top-k results. They are easy to interpret and directly correspond to user experience: if $\text{Hit}@5$ is 0.95, the correct document is within the top 5 results 95% of the time.

Mean Reciprocal Rank (MRR) captures ranking quality more precisely. For each query, the reciprocal rank is $1/\text{rank}$, where rank is the position of the first correct result. If the correct document is first, the reciprocal rank is 1.0. If it is second, it is 0.5. If it is fifth, 0.2. If it does not appear in the retrieved results at all, it is 0. MRR is the mean of these values across all queries. MRR penalises systems that retrieve the correct document but not at the top of the list — a property that $\text{Hit}@k$ does not capture. For a generative QA system, this matters because the language model is most strongly influenced by the documents at the top of its context window.

1.2.7 Observability in Software Systems

Software observability is the property that allows engineers to understand the internal state of a system by examining its outputs. The concept was popularised in the SRE (Site Reliability Engineering) community and is formalised around three pillars: traces (records of individual request journeys through distributed components), metrics (aggregated numerical measurements over time), and logs (structured event records).

In a typical RAG system, a single user query passes through multiple components: a document retrieval step that itself involves an embedding API call and a database query, followed by a generation step that involves another API call. Without distributed tracing, a developer observing that requests are slow has no way to know whether the bottleneck is in the embedding call, the vector search, or the generation step. With per-stage tracing, the answer is immediately visible.

OpenTelemetry (OTel) is the CNCF-graduated standard for distributed telemetry. It provides vendor-neutral APIs and SDKs for generating, collecting, and exporting traces, metrics, and logs in a consistent format. Traces exported via the OTLP protocol can be stored in any compatible backend (Grafana Tempo, Jaeger, Zipkin) and visualised as waterfall charts showing the timing and attributes of each span.

1.3 Problem Statement

When the work for this project began, the starting observation was simple: it is easy to build a RAG chat system that looks impressive in a demo, and it is very hard to know whether it is actually working well. This is not a criticism of the technology — RAG genuinely works — but of the surrounding engineering culture, which has mostly stopped at getting a working demo without thinking seriously about what happens after deployment.

The specific problems that this project was designed to solve can be stated as follows.

Problem 1: Retrieval quality changes invisibly. Any change to a RAG system — ingesting a new batch of documents, modifying the chunk size, switching embedding models, adjusting the number of retrieved results — can change retrieval quality for better or worse. These changes do

not throw errors, do not fail tests, and do not produce any warning. The chat interface continues to return answers that look plausible. The team does not know that quality has degraded until a user points it out, by which time the cause is often unclear.

Consider a concrete scenario that the project's regression case study demonstrates: a team ingests four new "overview" documents that contain broad summaries covering many topics. Each of these documents is textually close to a large number of user questions — it mentions every keyword but provides only a shallow treatment of each. These documents begin appearing in the top-k retrieved results for many queries, pushing the specific, authoritative source documents down in the ranking. The LLM now receives a diluted context and produces less precise answers. Nothing breaks. Tests pass. The change ships.

Problem 2: There is no continuous measurement. Most RAG teams compute retrieval metrics (if at all) during initial development, against a manually assembled set of test questions, and never revisit these measurements. As the corpus grows, as documents are modified, as pipeline parameters are tuned, the measurements from initial development become increasingly irrelevant. There is no equivalent of the automated test suite that runs on every pull request and catches regressions before they ship.

Problem 3: When quality does regress, diagnosis is difficult. Even if a team notices that retrieval quality has degraded, tracing the cause typically requires significant manual effort. Which questions were affected? Which documents are being incorrectly retrieved? Was it a specific change to the chunking strategy, or was it the new batch of documents? Without tooling that records per-question results keyed by stable identifiers, answering these questions is slow and error-prone.

Problem 4: Production observability is insufficient. A typical RAG deployment logs the question and the answer, and perhaps a total latency figure. This is not enough to operate the system with confidence. When a user reports a slow or incorrect response, there is no trace showing which stage was slow. There is no way to see that the embedding API took 2.5 seconds while the vector search took only 30 milliseconds. There is no cost tracking to know how much each query is spending. The system is a black box.

Problem 5: Evaluation and production are disconnected systems. In most setups, offline evaluation (running a test dataset through the retrieval pipeline) and online production traffic (real user queries) exist in completely separate places. A failing evaluation case and a related production query live in different tools and share no identifiers. This makes it impossible to look at a regression in CI and then find the production query that shows exactly why retrieval failed.

Problem 6: Quality is not a merge gate. Software teams routinely block pull requests when unit tests fail, type checks fail, or linting fails. Retrieval quality, however — arguably the most important measure of whether a RAG system is working — is never a merge gate. Bad changes ship freely because there is no automated mechanism to stop them.

Each of these six problems has a solution, and the proposed system implements all six of them. The evaluation harness solves problems 1 and 2. The run comparison view keyed by stable case IDs solves problem 3. OpenTelemetry tracing and the query-log explorer solve problem 4. The

shared database with a common query identifier solves problem 5. The CI regression gate solves problem 6.

1.4 Objectives of the Project

Based on the problem statement described above, the following objectives were defined for this project:

Objective 1: Design and implement a document-grounded chat interface that streams answers with inline citations, per-message cost tracking, and per-message latency display, supporting document ingestion in TXT, PDF, and DOCX formats.

Objective 2: Implement four distinct retrieval strategies — vector similarity, hybrid BM25-plus-vector, LLM-based reranking, and multi-query — each selectable per request and benchmarkable against the same evaluation dataset.

Objective 3: Build a persisted offline evaluation harness that computes Hit@1, Hit@3, Hit@5, Hit@8, and MRR on a fixed evaluation dataset, stores results with stable run identifiers in PostgreSQL, and exposes a frontend interface for listing and drilling into runs.

Objective 4: Implement a run comparison view that diffs two evaluation runs keyed by stable case IDs (not fragile row order), showing per-metric deltas and highlighting individual questions where the retrieved document changed between runs.

Objective 5: Implement a CI regression gate — a Python script with a deterministic exit code — that compares a candidate evaluation run to a pinned baseline and fails with exit code 1 when a gated metric (MRR or Hit@5) drops beyond a configured tolerance, blocking the pull request merge.

Objective 6: Instrument the RAG pipeline with OpenTelemetry distributed tracing at the per-stage level (embedding, vector search, reranking/multi-query, generation), export latency percentiles (p50/p95/p99) as Prometheus histograms, and visualise them in Grafana dashboards.

Objective 7: Build a query-log explorer that records every query with its retrieved chunks, latency breakdown, token counts, cost, trace ID, and optional evaluation run ID, enabling correlation of CI failures and live production traffic.

Objective 8: Containerise the full system with Docker Compose for single-command local deployment, and provide deployment guides for cloud platforms.

1.5 Scope of the Project

The scope of this project is deliberately focused on the quality-measurement and observability gap in RAG systems rather than on building a general-purpose AI platform. The following boundaries define what is and is not in scope.

In Scope:

The project covers the full lifecycle of a RAG system — from document ingestion through retrieval, generation, evaluation, and observability. This includes building the frontend chat interface and all observability pages, implementing the FastAPI backend with the complete RAG pipeline, designing and managing the PostgreSQL schema with pgvector, implementing all four retrieval strategies with their associated algorithms, building and validating the evaluation harness and CI gate, instrumenting the pipeline with OpenTelemetry, configuring Prometheus metrics and Grafana dashboards, and containerising the entire system. The evaluation case study demonstrating a real regression is also in scope.

Out of Scope:

This project does not build a hosted multi-tenant SaaS product. All data isolation is at the application level (per-user chat history), not at the database level. The project does not implement fine-tuning or training of any language model or embedding model. It does not build a mobile application. It does not implement real-time streaming evaluation (evaluation is batch/offline). Integration with third-party vector stores such as Pinecone, Weaviate, or Chroma is not covered — all vector storage uses pgvector within PostgreSQL. Production-grade security hardening (rate limiting, API key rotation, audit logging) is partially discussed in the `HARDENING.md` documentation file but is not the main focus of this project.

1.6 Existing System and Its Drawbacks

Several tools and platforms currently exist in the space of RAG evaluation and observability. This section examines the most significant ones and identifies their limitations.

1.6.1 LangSmith (LangChain)

LangSmith is a hosted platform developed by LangChain Inc. for tracing, testing, and evaluating LLM applications. It provides run tracking with a visual trace explorer, dataset management for regression testing, and a feedback collection mechanism. LangSmith is well-integrated with the LangChain framework and offers a polished user interface.

However, LangSmith has several significant limitations for the use case this project addresses. First, it is a proprietary cloud service, which means all traces and evaluation data are stored on LangChain's servers — a concern for organisations handling sensitive documents. Second, it requires adoption of the LangChain framework, which adds significant abstraction and makes it harder to understand what the system is actually doing at the level of individual API calls. Third, it does not provide a built-in CI regression gate — the concept of a merge-blocking quality check is not a native feature. Fourth, there is no shared identifier that connects an offline evaluation failure to a specific live production query.

1.6.2 Ragas

Ragas is an open-source Python library for evaluating RAG pipelines. It computes a suite of metrics including faithfulness (does the answer contradict the retrieved context?), answer relevance (does the answer address the question?), and context precision (how relevant are the retrieved chunks?). These metrics are computed using an LLM as a judge, making them expensive but potentially more aligned with human judgement than pure retrieval metrics.

Ragas is valuable as an evaluation library but is not a complete system. It does not provide a chat interface, a query log, a persistence layer for evaluation results, a comparison view, or a CI gate. Using Ragas requires substantial custom infrastructure to do what this project provides out of the box. Additionally, Ragas's reliance on an LLM judge for every evaluation case makes it significantly more expensive to run continuously — roughly a dollar or more per evaluation run on a modestly-sized dataset.

1.6.3 Arize Phoenix

Arize Phoenix is an open-source ML observability platform that has expanded to support LLM tracing and evaluation. It offers trace visualisation, dataset management, and some evaluation capabilities. Phoenix can run locally, which addresses the data privacy concern.

The main limitation of Phoenix in this context is that it is a standalone observability tool, not an integrated RAG application. Connecting Phoenix to a RAG system requires significant custom integration work. It does not provide the chat interface, the CI gate, or the shared database design that this project uses to connect evaluation failures and production traces.

1.6.4 TruLens

TruLens (developed by TruEra) is an open-source evaluation framework that implements the “RAG triad” — groundedness, context relevance, and answer relevance — as evaluation metrics. It integrates with several LLM frameworks and can log traces.

Like Ragas, TruLens is a library, not a complete deployable system. It also relies on an LLM judge for its core metrics, making it expensive for continuous evaluation. It does not provide a production chat interface, a CI gate, or a query-log explorer.

1.6.5 Common Drawbacks of All Existing Systems

Looking across all of these tools, several common limitations emerge:

First, none of them provide an integrated, self-contained system where the chat interface, evaluation harness, query log, and traces all share the same database and the same query identifier. In all existing tools, evaluation data and production traffic are separate systems that require custom integration work to connect.

Second, none of them implement a merge-blocking CI regression gate as a first-class feature. The concept of blocking a pull request when retrieval quality drops is simply not something any existing tool provides out of the box.

Third, the tools that are general enough to address these needs (LangSmith, Phoenix) are either proprietary cloud services or standalone tools that require significant custom integration. There is no existing open-source, self-hostable, full-stack RAG system that includes all of evaluation, observability, CI gating, and a production chat interface in a single deployable repository.

1.7 Proposed System

The proposed system addresses each of the drawbacks identified above through a unified, self-contained architecture where every component — chat, ingestion, evaluation, tracing, metrics, query logging — reads from and writes to the same PostgreSQL database, and every query (whether from a real user or from the evaluation harness) shares a common query-log row with a stable identifier.

The system is structured around four core capabilities:

Core Capability 1 — Grounded Chat with Per-Message Observability

The frontend provides a streaming chat interface built on Next.js 15 and React 19. Users can upload documents in TXT, PDF, or DOCX format, which are chunked, embedded, and stored in PostgreSQL with pgvector. When a user asks a question, the system streams back the answer token by token, with inline citations showing which document chunks the answer came from. Below each answer, a panel shows the total latency, the latency breakdown by pipeline stage, the token count, the cost in USD, and a link to the query-log row — which in turn links to the distributed trace in Grafana Tempo.

Users can select from four retrieval strategies: vector similarity, hybrid search, reranking, and multi-query. Each strategy can be evaluated independently using the evaluation harness, allowing data-driven selection of the best strategy for a given corpus and query type.

Core Capability 2 — Persisted Evaluation with Run Comparison

The offline evaluation harness (`eval/run_eval.py`) runs a fixed dataset of 78 question-answer pairs through the retrieval pipeline and computes Hit@1, Hit@3, Hit@5, Hit@8, and MRR. Unlike typical eval scripts that print results to the console and are forgotten, this harness persists every run to PostgreSQL — one row in `eval_runs` for the summary metrics, and one row in `eval_case_results` for each individual question's result.

The frontend eval pages let developers list all persisted runs, drill into any single run to see per-case results, and compare two runs side by side. The comparison is keyed by the `case_id` field — a stable string identifier from the dataset — rather than by database row order. This matters because datasets are frequently modified (new cases added, cases removed, order changed), and a comparison keyed on row order would silently produce nonsense deltas after any such modification.

The comparison view shows each gated metric's delta, a pass/fail verdict, and an expandable per-case table showing exactly which questions flipped from hit to miss (or vice versa) between the two runs.

Core Capability 3 — CI Regression Gate

The comparison script (`eval/compare_eval.py`) is designed to run in CI (GitHub Actions, GitLab CI, or any equivalent) as a step in the pull request check pipeline. It takes two run IDs as arguments, retrieves the corresponding runs from the database, diffs the gated metrics (Hit@5 and MRR), posts a formatted delta table to the pull request as a comment, and exits with code 0 if no gated metric has regressed beyond tolerance, or code 1 if any gated metric has regressed. A non-zero exit code fails the CI check and blocks the merge.

The tolerance values — 2 percentage points for Hit@5, 0.02 for MRR — are configurable and are set conservatively to avoid failing on natural run-to-run variance (which is typically less than 0.5 percentage points for a 78-case dataset). The baseline run is pinned in the repository as a CI artifact rather than being automatically promoted, giving the team deliberate control over what constitutes an acceptable baseline.

Core Capability 4 — Full-Pipeline Observability

The FastAPI backend instruments every stage of the RAG pipeline using the OpenTelemetry Python SDK. Each request produces a tree of spans: a root span for the entire request, a child span for the retrieval stage (which itself contains child spans for the embedding call and the vector search), and a sibling span for the generation stage (which contains a child span for the LLM call). Every span carries attributes including the model name, the number of tokens used, and the latency in milliseconds.

These spans are exported via OTLP to Grafana Tempo and are visible in Grafana as waterfall charts. The same trace ID is injected into the structured log entry for the request and into the query-log row in PostgreSQL, creating three-way correlation between traces, logs, and the query-audit database.

The backend also exports Prometheus-format histograms for request latency per route and per pipeline stage, allowing p50/p95/p99 latency percentiles to be computed and visualised in Grafana dashboards. A one-command Docker Compose profile brings up the full observability stack (Tempo, Prometheus, Grafana) with pre-configured datasources and pre-built dashboards.

1.8 Advantages of the Proposed System

Advantage 1: Retrieval quality is measured continuously, not just once. By running the evaluation harness against a fixed dataset on every pull request, the system transforms retrieval quality from a static measurement taken at initial development into a live, continuously updated metric that reflects the current state of the system. Teams can see immediately whether a proposed change improves or degrades retrieval — the same way they see whether it breaks unit tests.

Advantage 2: Regressions are attributable to specific questions and documents. When a run comparison shows that MRR dropped, the developer does not just see an aggregate number — they can expand the per-case view and see exactly which 12 questions had their correct source

pushed out of the top rank, and which incorrect document replaced it. This makes debugging a regression a matter of minutes rather than hours.

Advantage 3: Quality regressions are merge-blocking. The CI gate provides a hard enforcement mechanism. A change that degrades retrieval beyond tolerance cannot be merged without deliberate action to adjust the baseline or the tolerance. This shifts the team’s relationship with retrieval quality from passive observation to active enforcement — the same relationship they have with code correctness through unit tests.

Advantage 4: Latency is attributed to specific pipeline stages. When a user reports a slow query, the developer can open the trace in Grafana and see immediately whether the bottleneck was the embedding API call (typically 200–500ms), the vector search (typically under 50ms), or the generation LLM call (typically 1–3 seconds). This changes the diagnostic process from guesswork to measurement and points directly to the correct optimisation.

Advantage 5: Evaluation failures and production traffic share one mental model. Because both the evaluation harness and production user queries write to the same queries table with the same schema, and both produce OpenTelemetry traces with the same instrumentation, a developer looking at a failing eval case can look at the corresponding query-log row and, from there, open the distributed trace showing exactly how the retrieval pipeline processed that specific question. The transition from “this eval case is failing” to “here is why it failed in the trace” is a single click.

Advantage 6: The system is self-contained and self-hostable. Unlike cloud-based evaluation platforms, all data — documents, evaluation results, query logs, traces, metrics — remains within the operator’s own infrastructure. There are no third-party platform dependencies, no usage-based pricing for the evaluation system itself, and no data privacy concerns about evaluation data leaving the organisation’s network. The system can be deployed to any cloud provider or on-premises server with Docker.

Advantage 7: Cost is measured and visible. Every query records the number of prompt and completion tokens consumed and computes the cost in USD at current API rates. The evaluation harness reports the total cost of a run. The strategy benchmarking script reports cost per 1,000 queries for each strategy. This information, which is typically invisible in RAG systems, enables data-driven decisions about which retrieval strategy to deploy.

Advantage 8: The system is built with production-grade engineering practices. The codebase includes a CI pipeline that runs linting, type checking, unit tests, integration tests (against a real PostgreSQL instance), and Playwright end-to-end tests including accessibility checks. The backend has a minimum 70% test coverage requirement enforced in CI. Database schema changes are managed through versioned Alembic migrations (backend) and Drizzle migrations (frontend). The system is containerised with health checks and startup ordering guarantees.

CHAPTER 2

LITERATURE SURVEY

2.1 Review of Related Work

2.1.1 The Foundational RAG Paper

The work that most directly motivates this project is Lewis et al. (2020), “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” published at NeurIPS 2020. This paper introduced the term “RAG” and formalised the retrieve-then-generate paradigm as a trainable end-to-end system. The authors combined Dense Passage Retrieval (DPR), a bi-encoder model for approximate nearest-neighbour search over document passages, with BART, a sequence-to-sequence language model, and showed that the combined system outperformed both retrieval-only and generation-only baselines on open-domain QA benchmarks including Natural Questions, TriviaQA, and WebQuestions.

The central contribution of this paper, beyond the specific model architecture, was the conceptual framing: augmenting a generative model with a retrieval component at inference time, rather than trying to bake all knowledge into the model’s parameters, is both more efficient and more flexible. The retrieved documents provide grounding for the generation, and the knowledge base can be updated without retraining the generative model. This insight is what makes RAG practical for real-world applications with dynamic, private, or domain-specific knowledge bases, and it is the conceptual foundation of this project.

2.1.2 Dense Passage Retrieval

Karpukhin et al. (2020), “Dense Passage Retrieval for Open-Domain Question Answering,” is the paper that introduced DPR as a retrieval system and demonstrated that dense retrieval can outperform BM25 on several open-domain QA benchmarks. DPR uses two separate BERT encoders — one for questions and one for passages — trained with a contrastive loss to push question-passage pairs for the same answer closer together and unrelated pairs farther apart in the embedding space.

The key lesson from this work for this project is that retrieval quality is strongly dependent on the quality and training of the embedding model. A well-trained embedding model that understands the domain of the documents will produce much better retrieval than a generic model applied without adaptation. This motivates the benchmarking harness in this project — the ability to swap embedding models and measure the effect on retrieval quality is one of the most important use cases for the evaluation infrastructure.

2.1.3 The Case for Hybrid Retrieval

Luan et al. (2021), “Sparse, Dense, and Attentional Representations for Text Retrieval,” conducted a systematic analysis of sparse retrieval (BM25), dense retrieval (DPR-style), and attention-based models across several IR benchmarks. Their finding — that hybrid approaches combining sparse and dense signals typically outperform either alone — is now widely accepted in the information retrieval community.

The intuition behind this is that sparse retrieval is very reliable for queries with specific technical terms, product names, or identifiers, where the exact string match matters. Dense retrieval handles paraphrase and concept-level similarity better. Neither alone is dominant across all query types, so a combination that leverages both tends to be more robust. The hybrid search strategy in this project implements this principle by combining BM25 scores (computed from the raw text) with vector similarity scores (computed from the pgvector embeddings) using Reciprocal Rank Fusion.

2.1.4 Reciprocal Rank Fusion

Cormack et al. (2009), “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” introduced RRF as a simple but surprisingly effective method for combining results from multiple retrieval systems. The key insight is that instead of trying to normalise scores across systems — which is difficult when the score distributions differ — RRF works only with ranks. Each document’s RRF score is the sum of $1/(k + \text{rank}_i)$ across all retrieval systems, where k is a smoothing constant (typically 60) that reduces the impact of very high ranks.

This approach has been reproduced as a strong baseline in many subsequent multi-retrieval-system evaluations. Its simplicity (no parameters to tune beyond k) and robustness (it is insensitive to the scale of individual retrieval systems’ scores) make it well-suited for production use. This project uses RRF as the fusion method in both the hybrid search strategy and the multi-query strategy.

2.1.5 Cross-Encoder Reranking

Nogueira and Cho (2019), “Passage Re-ranking with BERT,” demonstrated that a cross-encoder model — one that jointly encodes the query and each candidate document through a BERT transformer — significantly outperforms bi-encoder models when used as a second-stage reranker. The trade-off is computational: a cross-encoder must process each query-document pair independently, making it impractical for first-stage retrieval over a large corpus but very effective over a small candidate set.

The reranking strategy in this project adapts this idea using the generative language model (GPT-4o-mini) as the reranker rather than a dedicated cross-encoder model. The LLM is prompted to assign a relevance score to each of the top-20 candidates from the first-stage vector retrieval, and the top- k by LLM score are returned as the final result. This approach is more expensive than a dedicated cross-encoder reranker but requires no additional model deployment, making it practical for a project that already uses the OpenAI API.

2.1.6 Multi-Query Retrieval and Query Expansion

Mao et al. (2021), “Generation-Augmented Retrieval for Open-Domain Question Answering,” explored the idea of using a language model to augment or expand the query before retrieval. By generating related questions or rephrased versions of the original query and retrieving passages for each, the system can surface relevant documents that would not have been found by the original query alone. This is particularly useful for complex, multi-hop questions where no single query captures all the relevant aspects.

The multi-query strategy in this project applies this principle: the LLM generates three reformulations of the user’s query, each targeting a different aspect or phrasing of the question. Each reformulation is embedded and searched independently, and the results are merged using RRF. The benchmarking results in this project show that on the 78-case evaluation corpus, multi-query does not outperform vector similarity — an important finding that illustrates why benchmarking is necessary. The benefit of multi-query depends heavily on the distribution of query types and the quality of the corpus, and one should not assume it will help without measuring.

2.1.7 Evaluation Metrics in Information Retrieval

The evaluation metrics used in this project — Hit@k and MRR — are well established in the information retrieval literature. Voorhees and Harman (2005) in “TREC: Experiment and Evaluation in Information Retrieval” provide a thorough treatment of IR evaluation methodology, including the importance of stable test collections and reproducible experimental protocols. The TREC (Text REtrieval Conference) series, running since 1992, has been the primary venue for developing and validating IR evaluation practices.

MRR, in particular, has been used extensively since Voorhees (1999) as a metric for question answering systems, where it is important not just that the correct answer appears somewhere in the retrieved results but that it appears early enough in the list to influence the downstream system. For a RAG system where the retrieved chunks are provided to a language model as context, ranking quality is directly relevant: documents appearing first in the context window have a stronger influence on the generated answer than documents appearing later.

2.1.8 RAGAS: LLM-Based RAG Evaluation

Es et al. (2023), “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” introduced an evaluation framework that uses LLMs to compute metrics that go beyond retrieval quality to measure the quality of the generated answer as well. RAGAS computes three main metrics: faithfulness (does the answer contain only claims supported by the retrieved context?), answer relevance (does the answer address the question?), and context precision (what fraction of the retrieved context is relevant to the question?).

This paper is highly relevant as related work because it addresses the same evaluation gap that motivated this project. However, the approach differs in an important way: RAGAS requires LLM calls for every evaluation case (to assess faithfulness and relevance), making it expensive to run continuously in CI. The approach taken in this project — using retrieval-only metrics (Hit@k, MRR) that do not require an LLM judge — enables much cheaper continuous evaluation. The trade-off is that retrieval-only metrics do not capture answer quality directly, only whether the right documents were retrieved.

2.1.9 Hidden Technical Debt in ML Systems

Sculley et al. (2015), “Hidden Technical Debt in Machine Learning Systems,” is a highly influential paper from Google that argues ML systems accumulate technical debt in ways that are qualitatively different from and harder to manage than debt in conventional software. Among the specific categories of ML technical debt they identify are unstable data dependencies (retrieval quality

changing because the underlying corpus changed), evaluation and monitoring gaps (the absence of ongoing quality measurement), and the “CACE principle” — Changing Anything Changes Everything — which captures the way that any change to an ML pipeline can have cascading, unexpected effects on system behaviour.

Every one of the problems identified in Section 1.3 of this report is an instance of the technical debt patterns described by Sculley et al. This paper provides the theoretical framing for why the engineering infrastructure built in this project — continuous evaluation, merge gating, per-stage observability — is not over-engineering a simple problem, but necessary infrastructure for a system that involves an ML component. The key point is that ML systems require continuous measurement to be operated reliably, just as financial systems require continuous accounting.

2.1.10 The RAG Survey

Gao et al. (2023), “Retrieval-Augmented Generation for Large Language Models: A Survey,” provides a comprehensive taxonomy of RAG architectures developed since the original Lewis et al. paper. They categorise approaches into naive RAG (as described above), advanced RAG (pre-retrieval and post-retrieval processing, such as query rewriting, reranking, and context compression), and modular RAG (flexible pipeline components that can be combined in various ways).

The survey is useful for situating this project within the broader landscape. This project’s four retrieval strategies correspond to different points in the taxonomy: vector similarity is naive RAG, hybrid search is an advanced pre-retrieval technique, reranking is an advanced post-retrieval technique, and multi-query is a query expansion technique. By implementing all four and providing a shared evaluation harness to compare them, this project makes the trade-offs between these approaches empirically measurable rather than theoretical.

Table 2.1: Summary of Key Research Papers Reviewed

Paper	Authors	Year	Key Contribution	Relevance to Project
RAG for Knowledge-Intensive NLP	Lewis et al.	2020	Introduced RAG paradigm	Foundational architecture
Dense Passage Retrieval	Karpukhin et al.	2020	Bi-encoder dense retrieval	Vector similarity strategy
Sparse, Dense, Attentional	Luan et al.	2021	Hybrid retrieval superiority	Hybrid search strategy
Reciprocal Rank Fusion	Cormack et al.	2009	Score-free result fusion	Used in hybrid/multi-query
Passage Re-ranking with BERT	Nogueira & Cho	2019	Cross-encoder reranking	Reranking strategy
Generation-Augmented Retrieval	Mao et al.	2021	LLM query expansion	Multi-query strategy
TREC Evaluation Methods	Voorhees & Harman	2005	Hit@k, MRR metrics	Evaluation harness metrics
RAGAS	Es et al.	2023	LLM-judged evaluation	Related work comparison
Hidden Technical Debt in ML	Sculley et al.	2015	ML operational debt	Motivation for CI gate
RAG Survey	Gao et al.	2023	RAG taxonomy	Architecture classification

2.2 Study of Existing Technologies

2.2.1 FastAPI

FastAPI is a modern Python web framework developed by Sebastián Ramírez and first released in 2018. It is built on top of Starlette (for the ASGI server implementation) and Pydantic (for data validation and serialisation). FastAPI's defining characteristics are its use of standard Python type hints for automatic input validation and its native support for asynchronous request handling via Python's `asyncio`.

The choice of FastAPI for this project was made for several reasons. The RAG pipeline involves multiple I/O-bound operations: calls to the OpenAI embedding API, calls to the OpenAI chat API, and queries to the PostgreSQL database. Asynchronous handling allows the server to process other

requests during these waiting periods rather than blocking a thread, which significantly improves throughput under concurrent load. FastAPI's async support is first-class rather than bolted on, making it straightforward to write non-blocking pipeline code.

FastAPI also generates automatic interactive API documentation (Swagger UI and ReDoc) from the type annotations, which is useful during development for testing endpoints without a frontend. Its Pydantic integration ensures that all request and response data is validated against defined schemas, catching data shape errors at the boundary rather than deep in the pipeline.

Performance benchmarks consistently place FastAPI among the fastest Python web frameworks, comparable to NodeJS frameworks in throughput for I/O-bound workloads. For a RAG pipeline that spends most of its time waiting for API responses and database queries, this is directly relevant.

2.2.2 Next.js 15 and React 19

Next.js is a full-stack React framework developed and maintained by Vercel. Version 15, used in this project, introduces several significant capabilities. The App Router architecture (introduced in Next.js 13 and stabilised in subsequent versions) allows pages and layouts to be React Server Components by default, which means they render on the server and send only HTML to the client — reducing the JavaScript bundle size and improving initial page load performance.

React 19, also used in this project, introduces the `useTransition` and `useOptimistic` hooks for managing concurrent rendering, and the Server Actions feature for calling server-side code directly from client components without writing API routes. For this project, React 19's streaming capabilities are particularly important: the chat interface renders answer tokens as they arrive from the FastAPI backend's server-sent events stream, updating the DOM incrementally rather than waiting for the complete response.

The Vercel AI SDK (used in this project for the chat state management and streaming utilities) provides abstractions for streaming LLM responses to React components, handling the server-sent events protocol and updating the chat state in real time.

2.2.3 PostgreSQL and the pgvector Extension

PostgreSQL has been the project's database of choice for a combination of reasons that go beyond simple feature comparison. First, PostgreSQL supports the `pgvector` extension, which adds native vector data types and approximate nearest-neighbour search operations directly within the relational database. This eliminates the need for a separate vector store (such as Pinecone, Weaviate, or Chroma) and keeps all data — documents, chunks, embeddings, query logs, evaluation results — in a single, transactionally consistent database.

The `pgvector` extension adds a `vector(n)` column type that can store an n-dimensional floating-point vector. It supports three distance operators: cosine distance (`1 - cosine_similarity`, used in this project), L2 (Euclidean) distance, and inner product. For approximate search at scale, `pgvector` supports two index types: `IVFFlat` (inverted file with flat quantisation) and `HNSW` (Hierarchical Navigable Small World). `HNSW` generally provides better search performance (higher recall at

a given query speed) but requires more memory to build. For the corpus sizes relevant to this project (tens of thousands of chunks), either index type works well; this project uses IVFFlat for its lower build time during testing.

PostgreSQL's support for JSONB (binary JSON) columns is also used extensively in this project — for storing chunk metadata, retrieved source lists in query logs, and evaluation case results. JSONB allows flexible schema extension without requiring formal migrations for fields that are not needed in structured queries.

2.2.4 Drizzle ORM

Drizzle ORM is a TypeScript-first object-relational mapper for SQL databases, used in this project for the frontend's chat and authentication schema (users, chats, messages). Drizzle differs from most ORMs in that it is schema-first: the developer defines the database schema in TypeScript, and Drizzle generates both the SQL migrations and the TypeScript types from that definition. This ensures that the TypeScript types in the application are always in sync with the database schema.

For the backend's RAG-related tables (documents, chunks, queries, eval_runs, eval_case_results), Alembic is used for migrations — Alembic being the standard migration tool for SQLAlchemy-based Python projects. The two migration systems coexist in the same PostgreSQL database because they manage non-overlapping sets of tables.

2.2.5 OpenTelemetry

OpenTelemetry is the CNCF project that has become the de-facto standard for distributed tracing, metrics, and logging in cloud-native systems. The project provides language-specific SDKs (this project uses the Python SDK for the FastAPI backend) that expose a consistent API for creating spans, recording attributes, and exporting telemetry data.

The Python SDK integrates with FastAPI through the `opentelemetry-instrumentation-fastapi` package, which automatically creates a root span for each incoming HTTP request. Within the request handler, this project creates additional child spans for each RAG pipeline stage using a custom context manager that records the stage name, duration, and key attributes (model name, token counts, retrieved document count).

Spans are exported using the OTLP (OpenTelemetry Protocol) exporter, which sends trace data to any OTLP-compatible backend. In this project, the backend is Grafana Tempo — a distributed tracing system designed specifically to work with Grafana. Tempo stores the traces and provides a query interface that Grafana uses to display trace waterfalls.

One important design decision was to make the OTel instrumentation degrade gracefully when no tracing backend is configured. The code uses a zero-cost no-op tracer (the default OTel tracer when no exporter is configured), meaning the application can be run without the observability stack and will function correctly — just without traces being collected.

2.2.6 Prometheus and Grafana

Prometheus is an open-source monitoring system originally developed at SoundCloud and now graduated as a CNCF project. It operates on a pull model: a Prometheus server periodically scrapes a `/metrics` endpoint on the application, ingesting the current values of all defined metrics. The application uses the `prometheus-client` Python library to define and maintain metric values — counters, gauges, histograms, and summaries.

This project uses Prometheus histograms for all latency metrics. A histogram accumulates observations (request durations) into configurable buckets and exposes three metric series: the count of observations, the sum of all observed values, and the count of observations falling into each bucket. From these, Prometheus can compute any quantile (p50, p95, p99) using the `histogram_quantile()` function. The advantage over a summary metric is that quantiles can be computed over arbitrary time ranges and across multiple server instances after the fact — a summary computes quantiles at collection time and cannot be aggregated across instances.

Grafana is the visualisation layer. It connects to Prometheus as a data source and allows developers to write PromQL (Prometheus Query Language) queries that are rendered as time-series graphs. This project ships pre-built Grafana dashboards as JSON files in the repository, so bringing up the observability stack provides immediately useful latency and cost dashboards without any manual configuration.

2.2.7 Docker and Docker Compose

Docker is the containerisation platform that allows the application and all its dependencies to be packaged into reproducible, isolated runtime environments. Each component — the PostgreSQL database, the FastAPI backend, the Next.js frontend, Grafana Tempo, Prometheus, and Grafana — runs in its own container with a defined image, environment variables, and network connectivity.

Docker Compose is the tool that orchestrates these containers, defining their relationships, dependencies, startup order, and network topology in a single `docker-compose.yml` file. This project uses Docker Compose profiles to allow different subsets of the stack to be started independently — the “full” profile starts everything, while starting only the database and backend is useful during frontend development with hot reloading.

2.2.8 Auth.js (NextAuth.js v5)

Auth.js, the TypeScript authentication library used in this project’s frontend, provides session management, credential-based login (email/password), and guest session support. Guest sessions — where a temporary user account is created automatically on first visit without requiring any login — are particularly useful for demos and evaluation, where requiring registration would create friction.

Session data is stored in PostgreSQL through Auth.js’s Drizzle adapter, which creates the necessary tables (accounts, sessions, verification_tokens, users) and handles the session lifecycle. The JWT signing key (`AUTH_SECRET`) is the only secret required for this functionality.

2.2.9 OpenAI API

This project uses two OpenAI APIs. The Embeddings API (model: text-embedding-3-small) converts text into 1,536-dimensional dense vectors. text-embedding-3-small was chosen over the larger text-embedding-3-large for its favourable cost-to-performance ratio — on the 78-case evaluation corpus, the quality difference between small and large is negligible while the cost difference is approximately 5×.

The Chat Completions API (model: gpt-4o-mini) is used for answer generation and, in the reranking strategy, for scoring candidate chunks. gpt-4o-mini provides a good balance of quality, speed, and cost for both use cases. The chat generation is streamed — using the stream=True parameter — which allows the frontend to begin displaying the response to the user before generation is complete.

The OpenAI client in this project wraps the official openai Python library with a custom client class that tracks token usage per call, computes cost, and injects the values into the OpenTelemetry span attributes and the query-log row.

2.3 Comparison of Previous Systems

Having reviewed the existing tools and the relevant research, it is worth stepping back and summarising how this project relates to and differs from the existing landscape. Table 2.2 presents a structured comparison.

Table 2.2: Comparison of Existing RAG Systems and Tools

Feature	This Project	LangSmith	Ragas	Arize Phoenix	TruLens
Integrated RAG Chat UI	Yes	No	No	No	No
Document Ingestion (PDF/DOCX/TXT)	Yes	No	No	No	No
Multiple Retrieval Strategies	Yes (4)	No	No	No	No
Persisted Eval Runs in DB	Yes	Yes	No	Partial	No
Run Comparison by Case ID	Yes	Partial	No	No	No
Merge-Blocking CI Regression Gate	Yes	No	No	No	No
Per-Stage OTel Tracing	Yes	Partial	No	Yes	No
Prometheus Metrics Export	Yes	No	No	Partial	No
Query Log with Trace Correlation	Yes	Partial	No	Partial	No
Self-Hosted / Open Source	Yes	No (cloud)	Yes	Yes	Yes
Single Deployable Repo	Yes	No	No	No	No
Per-Query Cost Tracking	Yes	Partial	No	No	No
Shared DB for Eval + Production	Yes	No	No	No	No
Docker Compose Deployment	Yes	No	No	No	No

The most distinctive aspect of this project is the last row in the table: a shared database that stores both offline evaluation results and online production traffic, connected by the same query identifier. No other tool in this space provides this property. The consequence is that the gap between “this eval case is failing” and “here is the production trace explaining why” — a gap that can take hours to bridge with existing tools — is eliminated by design.

The second most distinctive aspect is the merge-blocking CI gate. While some platforms provide diff-based comparison between runs, none of them provide the out-of-box mechanism for blocking a pull request merge when a gated metric regresses. This is the most operationally significant feature of the proposed system, and it is the one most likely to change the team’s day-to-day working relationship with retrieval quality.

CHAPTER 3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Requirement Analysis

Requirement analysis is the process of identifying, documenting, and validating the needs that a proposed system must satisfy. Good requirement analysis distinguishes between what the system must do (functional requirements) and the constraints under which it must do it (non-functional requirements), and it grounds both in a realistic assessment of whether the proposed system is achievable within the available constraints (feasibility study).

For this project, the requirement analysis was driven by two sources. The first was the problem statement developed in Chapter 1 — six specific, concrete problems in the operation of RAG systems, each of which translates directly into a system capability. The second was the study of existing systems in Chapter 2, which identified what is missing from current tools and what capabilities need to be added.

The requirements are organised around the system’s four core capabilities: the RAG chat interface, the evaluation harness and run comparison, the CI regression gate, and the pipeline observability. Cross-cutting concerns — authentication, containerisation, deployment, API design — are represented in the non-functional requirements.

One important characteristic of this requirement set is that many of the most important requirements are negative: the system must not require a third-party cloud platform, must not lose evaluation results when the harness script exits, must not produce comparison results that are invalidated by adding new cases to the dataset. These “must not” requirements reflect lessons learned from the limitations of existing systems.

3.2 Functional Requirements

The functional requirements define what the system must do from a user’s or developer’s perspective. Each requirement is stated at a level of specificity that makes it testable.

Table 3.1: Functional Requirements

FR ID	Requirement Description	Priority	Source
FR-01	The system shall accept file uploads in TXT, PDF, and DOCX formats and ingest them into the RAG knowledge base	High	Core feature
FR-02	The system shall segment documents into overlapping chunks with sizes adapted to document length	High	Core feature
FR-03	The system shall generate 1,536-dimensional vector embeddings for all chunks using the OpenAI text-embedding-3-small model	High	Core feature
FR-04	The system shall store chunks with their embeddings in PostgreSQL using the pgvector extension	High	Core feature
FR-05	The system shall retrieve relevant chunks using a vector similarity strategy (cosine nearest-neighbour search using pgvector)	High	Core feature
FR-06	The system shall retrieve relevant chunks using a hybrid strategy combining BM25 keyword scoring and vector similarity with Reciprocal Rank Fusion	High	Core feature
FR-07	The system shall retrieve relevant chunks using a reranking strategy that applies LLM relevance scoring over a candidate set from first-stage retrieval	High	Core feature
FR-08	The system shall retrieve relevant chunks using a multi-query strategy that generates query reformulations and merges results with RRF	High	Core feature
FR-09	The system shall stream grounded answers token-by-token from the backend to the frontend via server-sent events	High	Core feature
FR-10	The system shall display inline citations in the answer showing the source document for each retrieved chunk	High	Core feature
FR-11	The system shall record per-message latency, token count (prompt and completion), and cost in USD in the query audit log	Medium	Observability
FR-12	The system shall run an offline evaluation harness against a fixed JSONL dataset and compute Hit@1, Hit@3, Hit@5, Hit@8, and MRR	High	Evaluation
FR-13	The system shall persist each evaluation run in PostgreSQL as a summary row (eval_runs) with a UUID run ID	High	Evaluation

FR ID	Requirement Description	Priority	Source
FR-14	The system shall persist per-case results for each evaluation run in the <code>eval_case_results</code> table, keyed by stable <code>case_id</code> strings	High	Evaluation
FR-15	The system shall provide a frontend page listing all persisted evaluation runs with their metric summaries	Medium	Evaluation
FR-16	The system shall provide a frontend page for drilling into a single evaluation run showing per-case results	Medium	Evaluation
FR-17	The system shall provide a frontend page for comparing two evaluation runs side by side, keyed by <code>case_id</code> , with per-metric deltas	High	Evaluation
FR-18	The system shall provide a CI comparison script that takes two run IDs and exits with code 0 (no regression) or code 1 (regression beyond tolerance)	High	CI Gate
FR-19	The system shall post a formatted delta table as a PR comment when the CI comparison script is run in a GitHub Actions context	Medium	CI Gate
FR-20	The system shall emit OpenTelemetry spans for each pipeline stage: <code>rag.retrieve</code> , <code>openai.embedding</code> , <code>db.vector_search</code> , <code>rag.generate</code> , <code>openai.chat</code>	High	Observability
FR-21	The system shall export Prometheus-format latency histograms for each route and each pipeline stage at <code>/metrics/prometheus</code>	Medium	Observability
FR-22	The system shall store the OTel <code>trace_id</code> in each query-log row in PostgreSQL for correlation	High	Observability
FR-23	The system shall provide a query-log explorer page showing all query audit rows with retrieved chunks, latency, cost, and trace ID	Medium	Observability
FR-24	The system shall support guest session authentication (automatic user creation on first visit) and email/password authentication	Medium	Auth
FR-25	The system shall export evaluation results as JSON and CSV via API endpoints	Low	Export

Explanation of Key Functional Requirements

FR-14 (Stable `case_id`): This requirement deserves special attention because it represents one of the most important design decisions in the system. A naive evaluation comparison would compare results by database row order — comparing the first result of run A with the first result of run B. This breaks silently whenever the evaluation dataset is modified: adding a new case at position 5

shifts all subsequent cases, making the comparison produce nonsense deltas. The `case_id` requirement ensures that comparison is always between results for the same question, regardless of how the dataset has been reorganised.

FR-18 (CI gate exit code): The use of exit codes (0 for pass, 1 for fail) is the standard Unix convention for CI system integration. A non-zero exit code causes the CI step to be marked as failed, which (when the step is marked as required) prevents the pull request from being merged. This is how unit test frameworks, linters, and type checkers integrate with CI — and the evaluation gate uses exactly the same mechanism.

FR-20 (Per-stage spans): The requirement for per-stage spans rather than a single request-level span is crucial for the observability value proposition. A single request-level span would tell you how long the entire request took. Per-stage spans tell you exactly which component caused the latency — a fundamentally different and much more actionable diagnostic.

3.3 Non-Functional Requirements

Non-functional requirements (NFRs) define the quality attributes and operational constraints of the system. They address how well the system performs its functions rather than what functions it performs.

Table 3.2: Non-Functional Requirements

NFR ID	Category	Requirement	Rationale
NFR-01	Performance	End-to-end query latency (p95) shall be under 10 seconds for the vector similarity strategy on a standard development machine	User experience — answers above 10 seconds feel broken
NFR-02	Performance	Vector search latency shall be under 100ms for corpora up to 50,000 chunks	pgvector with IVFFlat index should achieve this comfortably
NFR-03	Performance	The evaluation harness shall be able to process 78 cases in under 5 minutes with default settings	Practical for running in CI
NFR-04	Scalability	The FastAPI backend shall be stateless, allowing horizontal scaling by running multiple instances behind a load balancer	Enables cloud deployment scaling
NFR-05	Scalability	The PostgreSQL connection pool shall support at least 10 concurrent connections without degradation	Handles concurrent evaluation runs and user queries
NFR-06	Reliability	The OTEL instrumentation shall degrade to a zero-cost no-op when no tracing backend is configured, without affecting application functionality	Allows running without the observability stack
NFR-07	Reliability	The application shall handle OpenAI API errors (rate limits, timeouts) gracefully, returning an appropriate error message to the user rather than crashing	Production robustness
NFR-08	Security	The backend API base URL shall be stored server-side only (in Next.js API route handlers) and never exposed to the browser client	Prevents backend URL leakage
NFR-09	Security	User passwords shall be hashed using bcrypt with a salt before storage in the database	Standard password security practice

NFR ID	Category	Requirement	Rationale
NFR-10	Security	API keys for the backend (if configured) shall be injected by the Next.js proxy and never sent from the browser	Prevents API key exposure
NFR-11	Maintainability	All PostgreSQL schema changes for the RAG/evaluation tables shall be managed through versioned Alembic migrations	Ensures reproducible schema across environments
NFR-12	Maintainability	All PostgreSQL schema changes for the chat/auth tables shall be managed through versioned Drizzle migrations	Ensures reproducible schema across environments
NFR-13	Maintainability	The backend test suite shall maintain a minimum of 70% code coverage, enforced in CI	Ensures a quality floor for backend code
NFR-14	Portability	The complete system shall be deployable using Docker Compose with a single command	Simplifies deployment across environments
NFR-15	Portability	The system shall be deployable to Render and Azure Container Apps using the provided deployment guides	Supports practical cloud deployment
NFR-16	Usability	The chat interface shall stream response tokens progressively without requiring page reload	Real-time UX
NFR-17	Usability	The evaluation comparison view shall visually highlight cases where Hit@5 flipped (correct source newly in top-5 or newly outside top-5)	Makes regression diagnosis faster
NFR-18	Reproducibility	Given the same corpus, embedding model, evaluation dataset, and retrieval strategy, the evaluation harness shall produce results within the natural run-to-run variance of the embedding API	Scientific validity of comparisons

NFR ID	Category	Requirement	Rationale
NFR-19	Observability	The latency histograms shall use bucket boundaries [0.1, 0.25, 0.5, 1.0, 2.0, 5.0, 10.0] seconds to enable meaningful p50/p95/p99 computation	Practical bucket sizing for RAG workloads

3.4 Feasibility Study

3.4.1 Technical Feasibility

Technical feasibility assesses whether the proposed system can be built using available technologies within a reasonable timeframe. The key question is whether the required capabilities exist in stable, well-documented tools.

Every component of the proposed system relies on mature, production-proven technology. FastAPI has been widely adopted in industry and has a large ecosystem of extensions. PostgreSQL is the world's most widely deployed open-source relational database, and the pgvector extension has seen rapid adoption in production deployments since its initial release. Next.js 15 is actively maintained by Vercel with a large community. OpenTelemetry has reached the CNCF Graduated stage (the highest maturity level) and is actively supported by all major cloud providers. The OpenAI APIs are production services with documented uptime SLAs.

The main technical risk identified during planning was whether pgvector would provide adequate search performance for the corpus sizes relevant to this project. Based on the pgvector documentation and published benchmarks, IVFFlat search over 100,000 vectors with 1,536 dimensions completes in under 5ms on a modern CPU. For the 78-case evaluation corpus used in this project, performance is not a concern at all.

A second technical risk was the integration between multiple observability components (OpenTelemetry, Tempo, Prometheus, Grafana). This was mitigated by using official Docker images with well-documented configuration and by implementing OTel instrumentation to degrade gracefully when the tracing backend is unavailable.

Table 3.3: Technical Feasibility Assessment

Component	Technology	Maturity Level	Risk Level
Backend API	FastAPI + Python 3.11	Production-proven (millions of deployments)	Low
Frontend	Next.js 15 + React 19	Production-proven (widely deployed)	Low
Vector Storage	PostgreSQL + pgvector 0.7	Rapidly adopted, stable API	Low
Database ORM (frontend)	Drizzle ORM 0.34	Stable, TypeScript-first	Low
Distributed Tracing	OpenTelemetry (CNCF Graduated)	Industry standard	Low
Trace Storage	Grafana Tempo	Production-proven	Low
Metrics	Prometheus + Grafana	Industry standard	Low
Auth	Auth.js v5	Stable beta	Medium
External API	OpenAI (embeddings + chat)	Production service with SLA	Low
Containerisation	Docker Compose v2	Industry standard	Low

Overall Technical Feasibility: High. All required capabilities exist in stable tools. No experimental or unproven technology is required. The most novel component is the integration of evaluation, CI gating, and observability in a single system — but each individual component is standard.

3.4.2 Economic Feasibility

Economic feasibility evaluates whether the proposed system can be built and operated within acceptable cost constraints. For an academic project, the cost analysis focuses on the ongoing operational costs that would apply during development and demonstration.

Development Phase Costs: During development, the system runs entirely locally using Docker. The only external cost is OpenAI API usage. A typical development session involving 50–100 queries and one or two evaluation runs costs approximately \$0.10–\$0.50 in API fees. Over a two-month development period with moderate usage, total development API costs were estimated at under \$20.

Evaluation Costs: A single evaluation run against the 78-case dataset using text-embedding-3-small (for embedding each question) and gpt-4o-mini (for generation, only in the reranking strategy) costs under \$0.01 for the vector similarity and hybrid strategies, and approximately \$0.08 for the reranking strategy. Running evaluations in CI on every pull request is therefore negligible in cost.

Deployment Costs: The system can be deployed on Render’s free tier for demonstration purposes at zero recurring cost. For a production deployment handling 1,000 queries per day, the primary cost is the OpenAI API: at approximately \$0.0002 per query for embeddings plus \$0.002 per query for gpt-4o-mini generation, the daily API cost is approximately \$2.20. All other system costs (server, database) would depend on the hosting provider but are comparable to any web application of similar complexity.

Table 3.4: Economic Cost Estimate

Item	Cost	Frequency
OpenAI API — Development Usage	~\$15	One-time (development period)
OpenAI API — Per Evaluation Run (vector/hybrid strategy)	~\$0.005	Per CI run
OpenAI API — Per Evaluation Run (reranking strategy)	~\$0.08	Per CI run
OpenAI API — Per 1,000 Production Queries	~\$2.20	Ongoing
Hosting — Render Free Tier	\$0	Monthly
Hosting — Basic Cloud Instance	~\$10–\$20	Monthly
All Open Source Software Licences	\$0	—

Overall Economic Feasibility: High. All software components are open-source and free to use. Operational costs are low and predictable, dominated by OpenAI API usage which scales linearly with query volume.

3.4.3 Operational Feasibility

Operational feasibility evaluates whether the proposed system can be practically deployed and used by its intended users — in this case, software developers building and maintaining RAG systems.

The system has been designed with operator experience as a first-class concern. Deployment requires only Docker Compose, a .env file with two required values (OPENAI_API_KEY and AUTH_SECRET), and a single command (docker compose –profile full up -d). The database migrations run automatically as part of the container startup sequence. The observability stack (Tempo, Prometheus, Grafana) can be started separately and is not required for the application’s core functionality.

For a developer maintaining the system, the daily workflow involves writing code, creating a pull request, and reviewing the CI gate results. If the gate fails, the comparison output includes the specific metric that regressed, the delta value, and — through the query-log explorer — a link to inspect the individual failing cases. This workflow adds approximately two minutes to the pull request review process for cases where the gate passes, and provides structured diagnostic information for cases where it fails.

The evaluation harness and comparison script are CLI tools that can be run manually at any time, not just in CI. This allows developers to run ad-hoc evaluations during development to check the effect of a local change before creating a pull request.

Overall Operational Feasibility: High. The system is designed for developer use, with minimal configuration requirements and a workflow that integrates naturally with standard software development practices.

3.4.4 Social and Legal Feasibility

The system processes documents provided by the user. The nature of those documents is entirely under the operator’s control. The system does not collect any data about users beyond what is necessary for authentication (email address and bcrypt-hashed password) and chat history. No analytics data is sent to third parties other than the OpenAI API calls required for embedding and generation.

The OpenAI API usage is subject to OpenAI’s Terms of Service. In the standard API terms, data submitted via the API is not used for model training unless the user opts in. This means that documents ingested into the system and queries submitted to it are not automatically used to train OpenAI’s models.

All software dependencies used in this project are released under permissive open-source licences (MIT, Apache 2.0, BSD). There are no GPL-licensed components that would impose copyleft restrictions on the project’s own code.

Overall Social and Legal Feasibility: High. No legal, ethical, or social barriers to deployment have been identified.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The architecture of this system was designed around one central constraint: every component — the chat interface, the document ingestion pipeline, the evaluation harness, the CI gate, and the observability layer — must share a single PostgreSQL database and must be able to look up any query by a common identifier. This constraint is what makes it possible to click from a failing CI evaluation case directly to the production trace that explains why retrieval failed. Without this shared data model, the system degrades into exactly the kind of disconnected tooling that the project was designed to replace.

The result is a three-tier architecture with an orthogonal observability layer. The three tiers are: the presentation layer (the Next.js 15 frontend serving the chat interface, the evaluation pages, the query-log explorer, and the metrics dashboard), the application layer (the FastAPI backend implementing the RAG pipeline, evaluation harness runner, and API endpoints), and the data layer (PostgreSQL with pgvector managing all persistent state). The observability layer collects telemetry from the application layer and routes it to separate storage backends — Grafana Tempo for traces, Prometheus for metrics — without affecting the main request path.

4.1.1 The Presentation Layer

The presentation layer is a Next.js 15 application using the App Router architecture. Pages are React Server Components by default, which means they render on the server and send HTML to the browser rather than shipping a large JavaScript bundle. Client-side interactivity (streaming chat, real-time token updates, interactive comparison views) is confined to specifically marked client components using the “use client” directive.

All requests from the browser to the FastAPI backend are proxied through Next.js API route handlers. This is not merely a convenience — it is a security requirement. The FastAPI backend URL (`BACKEND_API_BASE_URL`) is a server-side environment variable that is never included in the JavaScript bundle sent to the browser. The Next.js API route handler reads this variable on the server, forwards the request, and streams the response back to the browser. From the browser’s perspective, it is always talking to the same domain as the frontend, which also resolves CORS issues cleanly.

The frontend has five main page areas: the chat interface (the primary user-facing view), the evaluation run list and detail pages, the evaluation run comparison page, the query-log explorer, and the metrics dashboard. Each of these reads from the FastAPI backend, which in turn reads from PostgreSQL.

4.1.2 The Application Layer

The FastAPI backend is organised into four module groups. The API layer (`app/api/routes/`) contains the HTTP endpoint handlers. The RAG core (`app/rag/`) contains the pipeline logic: document ingestion, adaptive chunking, embedding generation, the four retrieval strategies, and answer generation. The database layer (`app/db/`) manages the connection pool and SQL interactions. The LLM layer (`app/llm/`) wraps the OpenAI client with token tracking and error handling. The core module (`app/core/`) contains configuration, tracing setup, and Prometheus metric definitions.

The backend is stateless — all persistent state lives in PostgreSQL, and any backend instance can handle any request. This makes horizontal scaling straightforward: multiple backend instances can run behind a load balancer without any session stickiness or inter-instance communication. The connection pool (managed by `asyncpg`) is configured per-instance.

4.1.3 The Data Layer

The data layer is PostgreSQL 15 with the `pgvector` extension. Two sets of tables coexist in the same database instance, managed by separate migration tools to keep concerns separated. The RAG and evaluation tables (`documents`, `chunks`, `queries`, `eval_runs`, `eval_case_results`) are managed by Alembic, the standard Python migration tool. The chat and authentication tables (`users`, `chats`, `messages`, `sessions`) are managed by Drizzle's migration system, which is TypeScript-native and integrates naturally with the frontend codebase.

The separation into two migration systems is a deliberate design choice: the RAG tables evolve with backend development (changes to the chunking schema, new evaluation fields), while the chat tables evolve with frontend development (new message metadata, chat organisation features). Keeping the migrations separate means a backend developer modifying the evaluation schema does not need to touch the Drizzle migration files, and vice versa.

4.1.4 The Observability Layer

The observability layer is not in the critical request path — it collects data from the application layer without blocking or slowing down request processing. The OpenTelemetry Python SDK in the backend creates spans using a context manager that wraps each pipeline stage. Spans are queued in memory and exported asynchronously via the OTLP exporter to Grafana Tempo. If Tempo is not running (e.g., during development without the observability stack), the no-op tracer is used and the spans are simply discarded without any error.

The Prometheus client in the backend maintains in-memory metric counters and histograms. A Prometheus server scrapes the `/metrics/prometheus` endpoint every 15 seconds and stores the values in its time-series database. Grafana reads from both Prometheus (for metrics dashboards) and Tempo (for trace waterfalls) through pre-configured datasources.

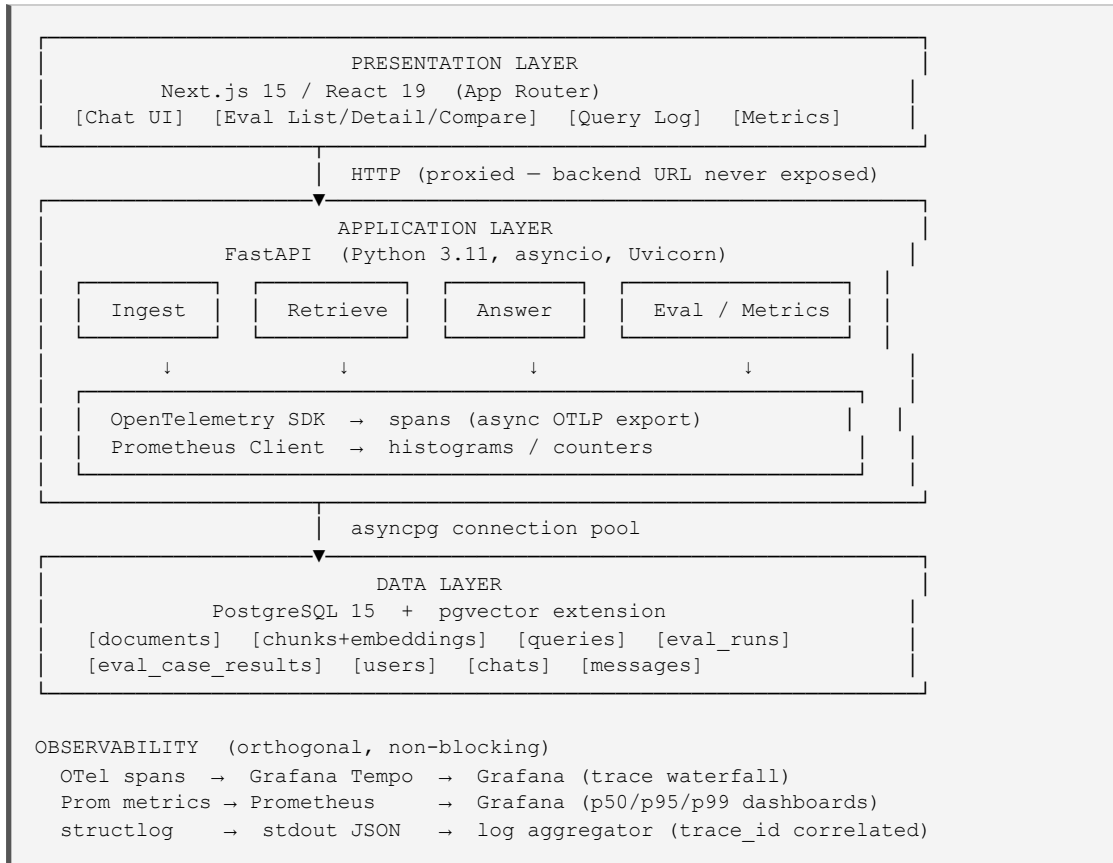


Figure 4.1: Three-Tier System Architecture with Orthogonal Observability Layer

4.2 UML Diagrams

4.2.1 Use Case Diagram

The use case diagram identifies the actors interacting with the system and the primary use cases each actor participates in.

Three actors interact with the system: the User (a person using the chat interface to ask questions about documents), the Developer/Operator (a team member monitoring system health, inspecting query logs, and reviewing evaluation results), and the CI System (GitHub Actions or equivalent, which runs the regression gate as part of the pull request pipeline).

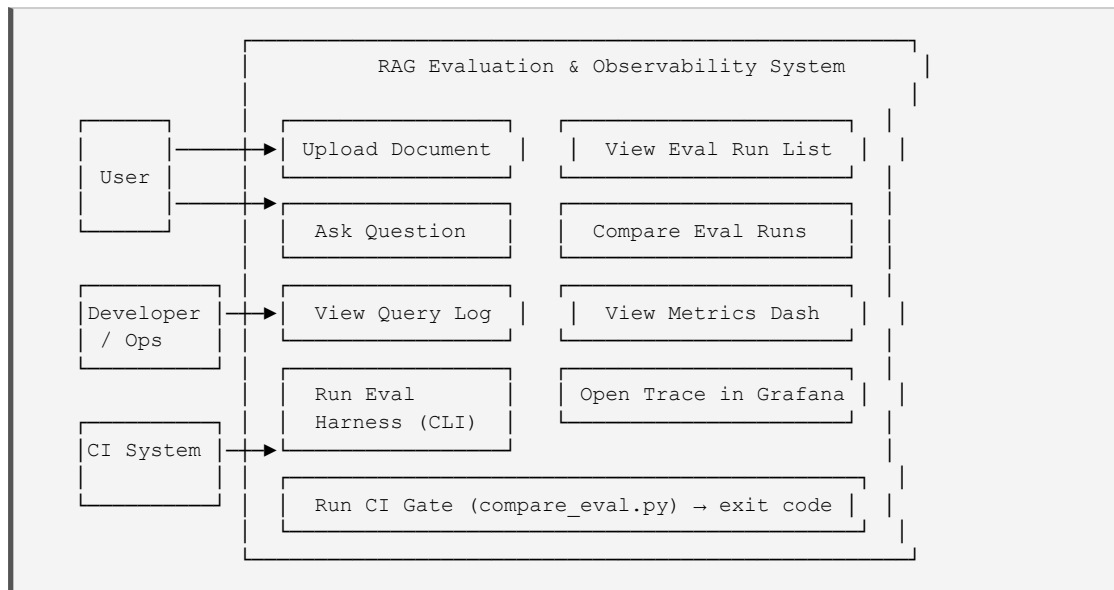


Figure 4.2: Use Case Diagram

4.2.2 Class Diagram — Backend RAG Modules

The class diagram below shows the key classes in the RAG pipeline and their relationships.

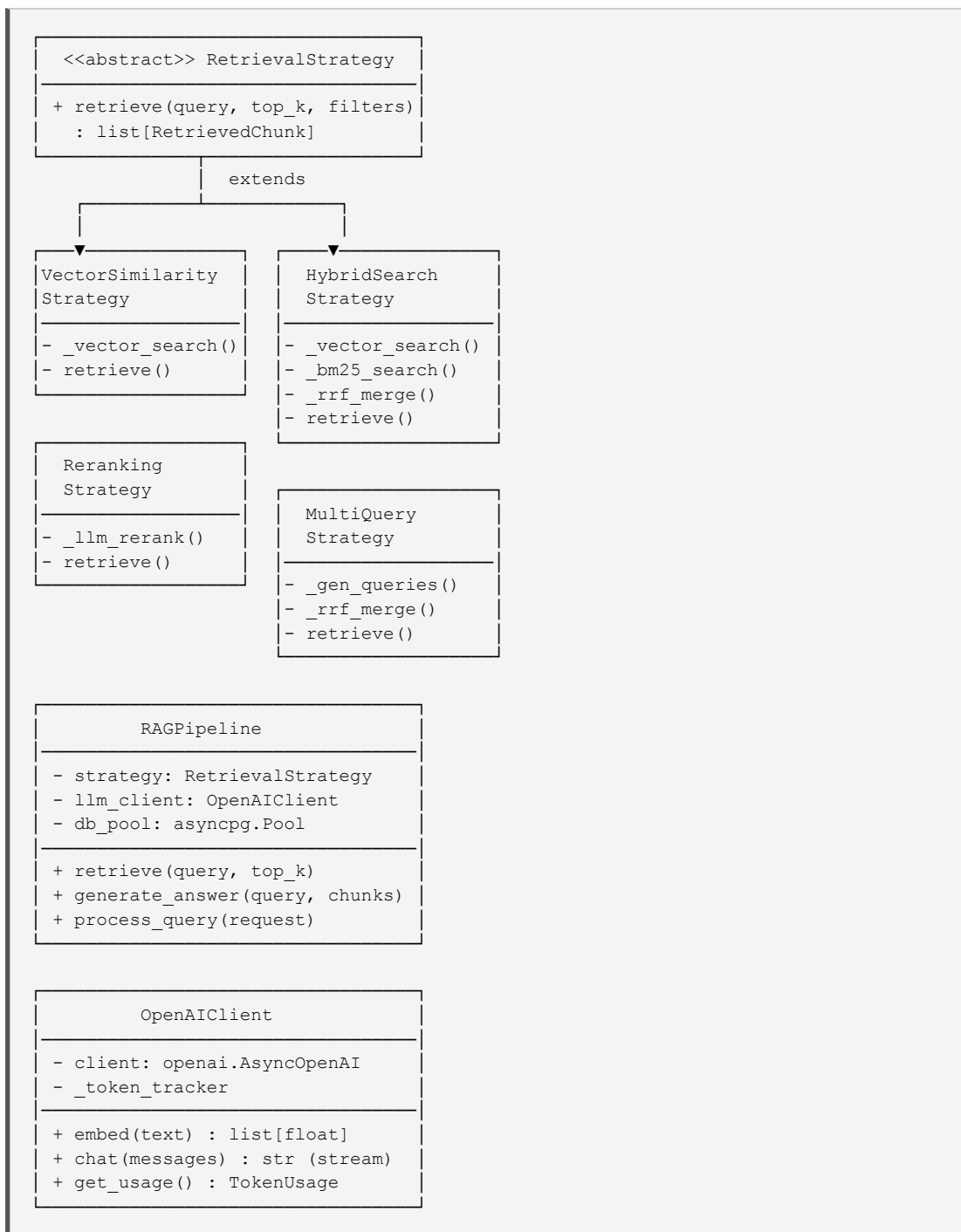


Figure 4.3: Class Diagram — Backend RAG Modules

4.2.3 Sequence Diagram — User Query Flow

This diagram traces the complete journey of a user query from the browser to the streamed response and query-log persistence.

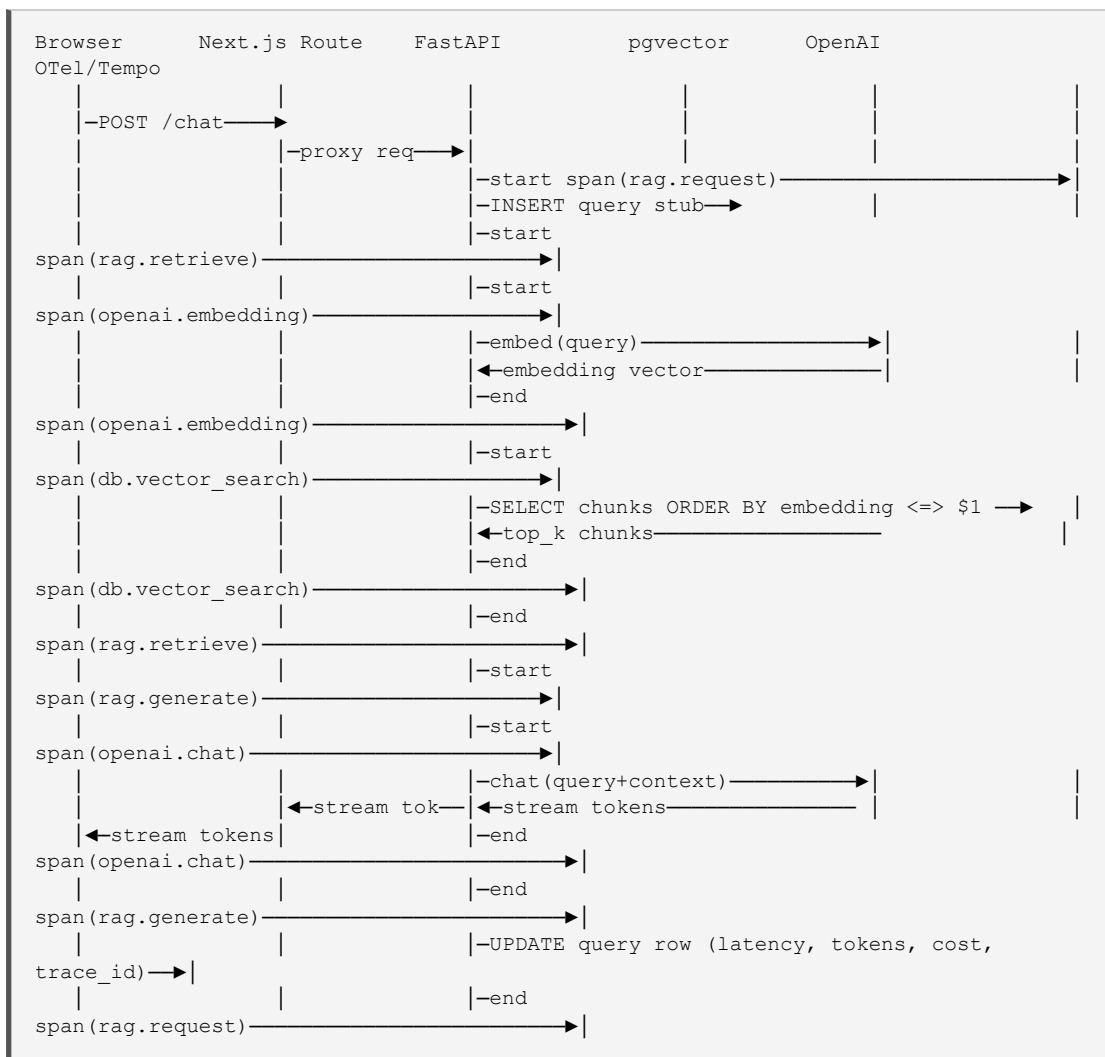


Figure 4.4: Sequence Diagram — User Query Flow

4.2.4 Sequence Diagram — Document Ingestion Flow

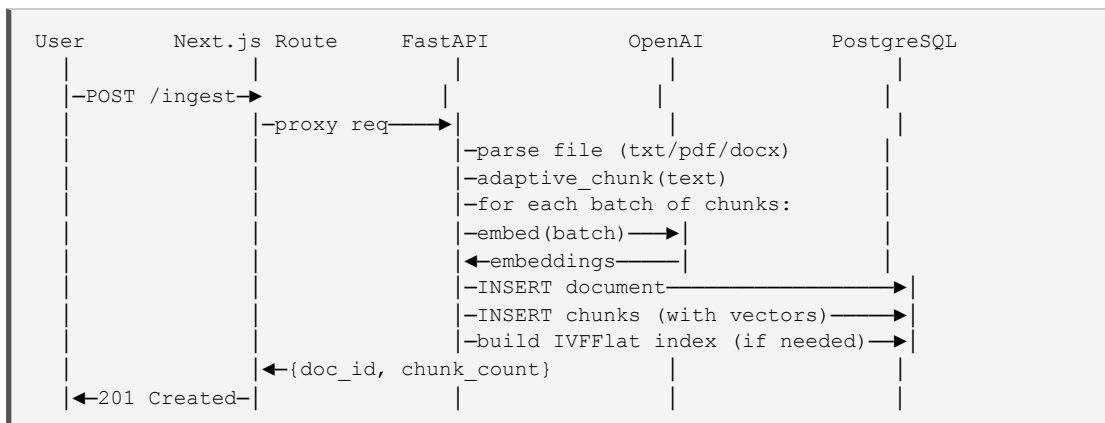


Figure 4.5: Sequence Diagram — Document Ingestion Flow

4.2.5 Sequence Diagram — Evaluation Run

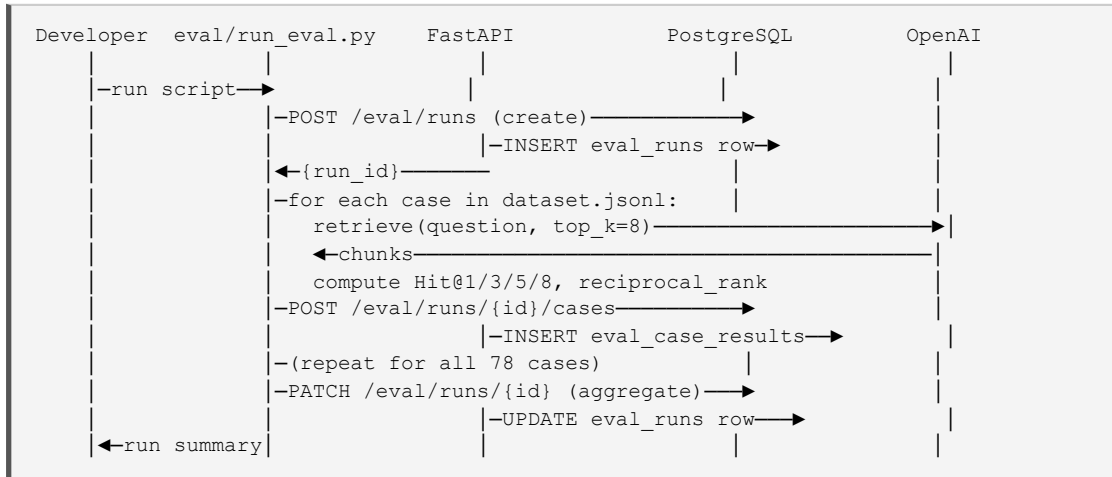


Figure 4.6: Sequence Diagram — Evaluation Run

4.2.6 Activity Diagram — CI Regression Gate

The activity diagram below shows the decision flow in the CI regression gate script (eval/compare_eval.py).

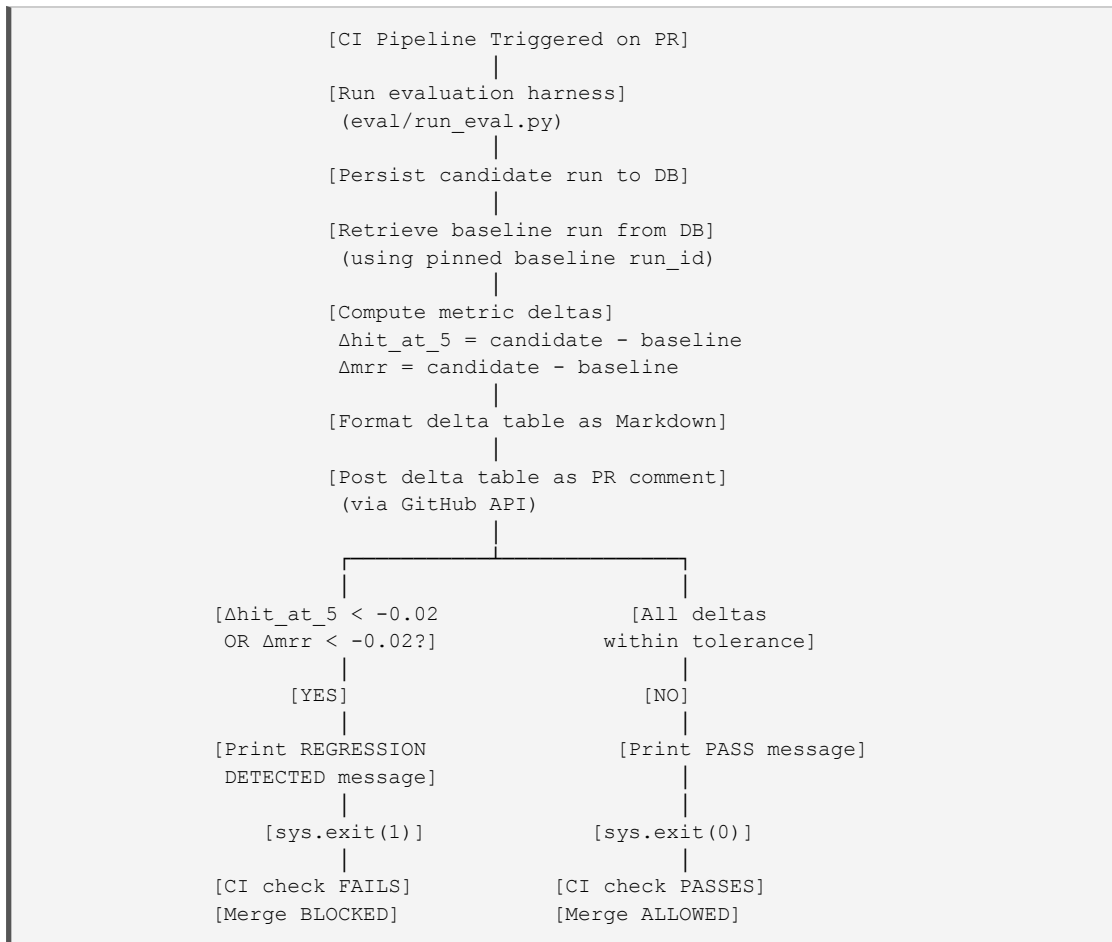


Figure 4.7: Activity Diagram — CI Regression Gate

4.2.7 Activity Diagram — Chat with Streaming and Citation Display

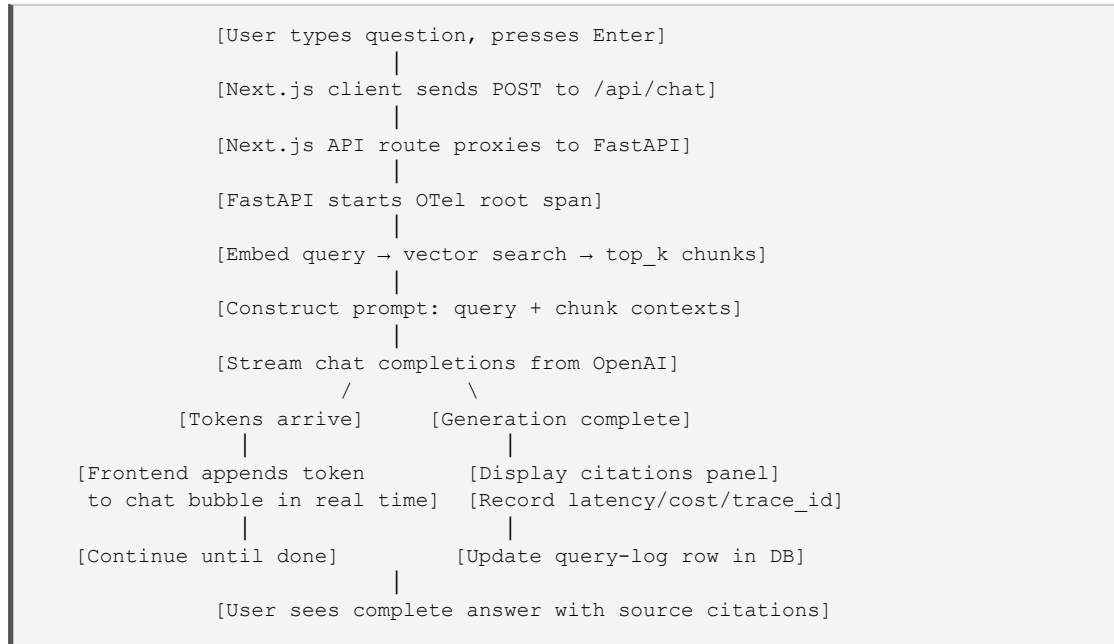


Figure 4.8: Activity Diagram — Chat with Streaming and Citation Display

4.3 Database Design

4.3.1 Entity Relationship Diagram

The database schema consists of two groups of tables. The RAG and evaluation schema (managed by Alembic) covers all aspects of the retrieval pipeline and evaluation. The chat and authentication schema (managed by Drizzle) covers user accounts, conversations, and messages.

The key relationships are as follows. Each document may have many chunks (one-to-many, with cascade delete). Each evaluation run may have many case results (one-to-many, with cascade delete). Each query-log row may optionally reference an evaluation run (many-to-one, nullable). Each chat belongs to one user, and each message belongs to one chat.

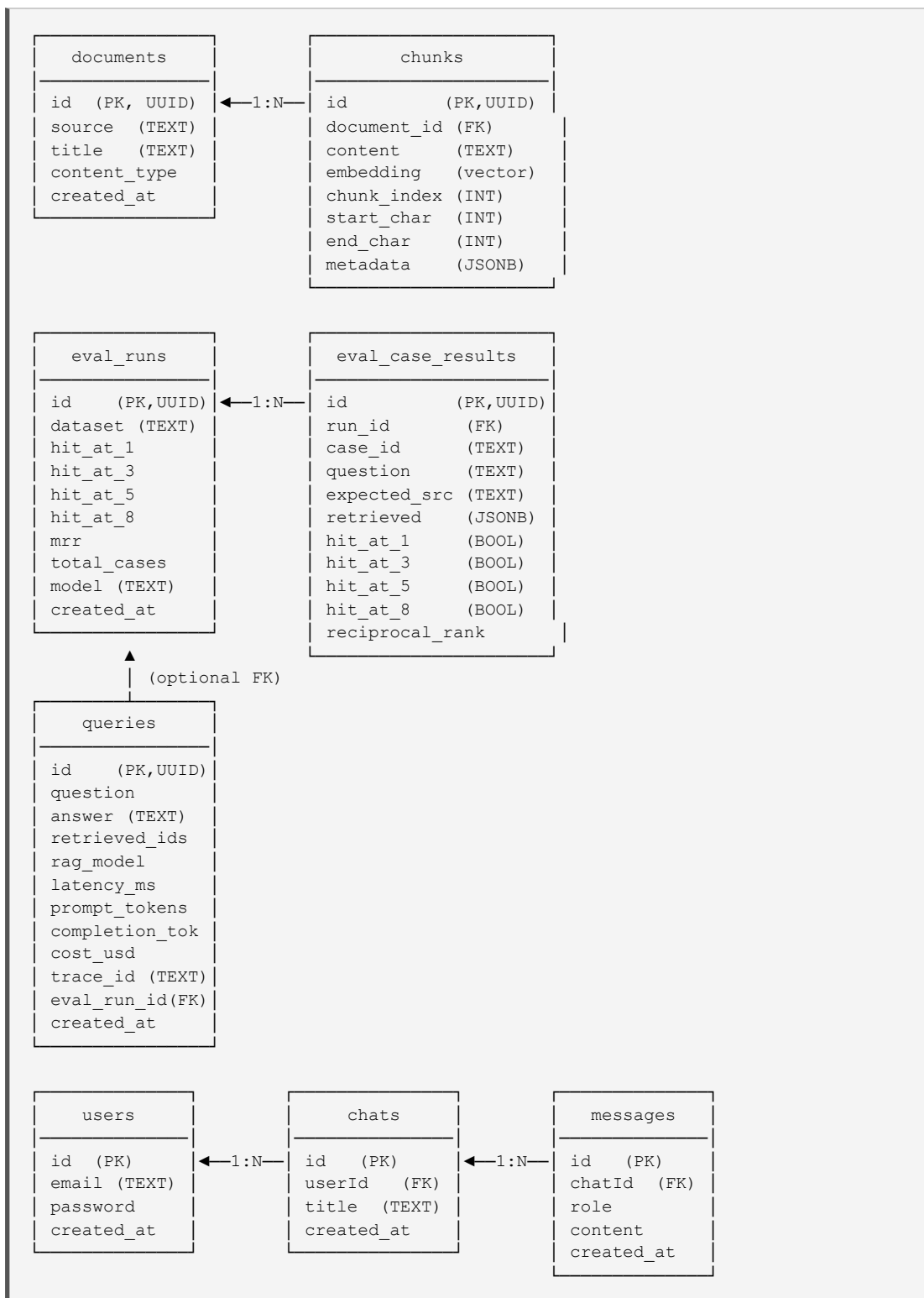


Figure 4.9: Entity Relationship (ER) Diagram

4.3.2 Index Strategy

Indexes are critical for query performance, especially for the vector similarity search which is the most computationally intensive operation. The following indexes are defined:

The chunks table has an IVFFlat index on the embedding column using vector_cosine_ops. This enables approximate nearest-neighbour search in $O(\sqrt{n})$ time rather than $O(n)$. The number of IVFFlat lists is set to 100 for corpora up to approximately 1 million vectors.

The eval_case_results table has a composite index on (run_id, case_id), which is the primary access pattern for the comparison query (fetch all results for two run IDs and join on case_id).

The queries table has an index on eval_run_id (for the query-log explorer's filter by evaluation run) and on created_at (for the default time-ordered listing).

4.4 Data Flow Diagrams

4.4.1 Level 0 — Context Diagram

The context diagram shows the system as a single process and identifies all external entities that interact with it.

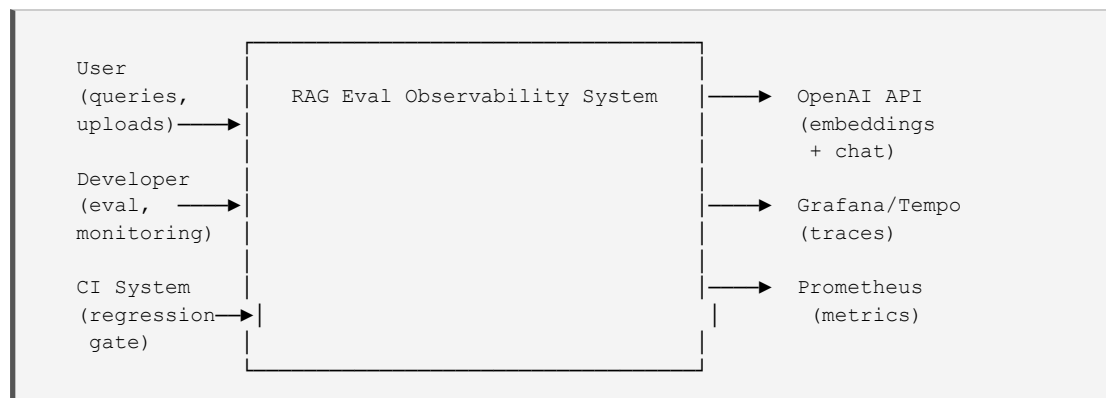


Figure 4.10: DFD Level 0 — Context Diagram

4.4.2 Level 1 — Main Data Flows



Figure 4.11: DFD Level 1 — Main System Data Flows

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Hardware Requirements

The system is designed to run comfortably on the hardware available to most university students and small development teams. The requirements listed below represent the minimum needed for a smooth development and testing experience.

Table 5.1: Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Dual-core 64-bit CPU (x86-64)	Quad-core CPU, 2.5 GHz+
RAM	4 GB	8 GB or more
Storage	10 GB free disk space	20 GB SSD
Network	Stable broadband connection	Broadband (for OpenAI API calls)
Operating System	Linux, macOS 12+, Windows 10 (with WSL2)	Ubuntu 22.04 LTS or macOS 13+

The RAM requirement of 4 GB is the minimum to run PostgreSQL, the FastAPI backend, and the Next.js frontend simultaneously in Docker containers. The 8 GB recommendation allows the full observability stack (Tempo, Prometheus, Grafana) to run alongside the application without memory pressure.

5.2 Software Requirements

Table 5.2: Software Stack — Backend

Software	Version	Purpose
Python	3.11 or later	Backend runtime
uv	0.4+	Python package and environment manager
FastAPI	0.115+	REST API framework
Uvicorn	0.32+	ASGI server
asyncpg	0.30+	Async PostgreSQL driver
Alembic	1.13+	Database migration tool
openai	1.50+	OpenAI Python SDK
opentelemetry-sdk	1.27+	Distributed tracing SDK
opentelemetry-exporter-otlp	1.27+	OTLP trace exporter
prometheus-client	0.21+	Prometheus metrics
structlog	24.4+	Structured logging
Pydantic	2.9+	Data validation
pytest	8.3+	Testing framework
pytest-asyncio	0.24+	Async test support
coverage	7.6+	Code coverage measurement

Table 5.3: Software Stack — Frontend

Software	Version	Purpose
Node.js	20 LTS	JavaScript runtime
pnpm	9+	Package manager
Next.js	15.5+	Full-stack React framework
React	19.0+	UI component library
TypeScript	5.6+	Type-safe JavaScript
Drizzle ORM	0.34+	TypeScript-first ORM
Auth.js	5.0 beta	Authentication library
Vercel AI SDK	5.0	Streaming AI utilities
Jest	29+	Unit testing
Playwright	1.48+	End-to-end browser testing
PostgreSQL	15+	Database (shared with backend)
pgvector	0.7+	Vector similarity extension
Docker	24+	Containerisation
Docker Compose	2.20+	Multi-container orchestration

5.3 Development Environment Setup

Setting up the development environment from scratch requires the following steps. These steps have been validated on macOS 13 and Ubuntu 22.04; Windows users should use WSL2.

Step 1 — Install prerequisites. Docker Desktop (which includes Docker Compose) must be installed. For native development (without Docker for the application itself, using Docker only for the database), Python 3.11+ with uv and Node.js 20+ with pnpm must also be installed.

Step 2 — Clone and configure.

```
git clone https://github.com/revanthchary04/rag-eval-observe.git
cd rag-eval-observe
cp .env.example .env
```

The `.env` file requires two mandatory values: `OPENAI_API_KEY` (obtained from the OpenAI platform dashboard) and `AUTH_SECRET` (a 32-byte random string, generated with `openssl rand -base64 32`). All other environment variables have sensible defaults.

Step 3 — Start the full stack.

```
docker compose --profile full up -d
```

This command starts all containers in dependency order: PostgreSQL first (with pgvector extension initialised), then the FastAPI backend (which applies Alembic migrations on startup), then the Next.js frontend (which applies Drizzle migrations on startup and loads the demo corpus). The health check endpoint at `http://localhost:8000/api/v1/health` confirms the backend is ready.

Step 4 — Seed the demo corpus. The demo corpus (five short RAG-topic documents that the evaluation dataset references) is seeded automatically by the frontend container's startup script. Running `pnpm seed:corpus` manually from the repo root re-seeds it idempotently (existing documents with the same source URL are skipped).

Step 5 — Optional: Start the observability stack.

```
docker compose --profile observability up -d
```

This starts Grafana Tempo, Prometheus, and Grafana. Grafana is available at `http://localhost:3001` with pre-configured datasources and dashboards.

Directory Structure:

```

rag-eval-observe/
├── backend/
│   ├── app/
│   │   ├── api/routes/      # HTTP endpoint handlers
│   │   ├── core/           # Config, tracing, metrics
│   │   ├── db/             # Database session, SQL helpers
│   │   ├── llm/            # OpenAI client wrapper
│   │   └── rag/            # Pipeline: ingest, chunk,
│   │       ├── ingest.py    # retrieve, answer
│   │       ├── chunking.py
│   │       ├── adaptive_chunking.py
│   │       ├── retrieve.py
│   │       ├── retrieval_strategies.py
│   │       ├── answer.py
│   │       └── types.py
│   ├── eval/
│   │   ├── run_eval.py      # Evaluation harness
│   │   ├── compare_eval.py  # CI regression gate
│   │   ├── benchmark_strategies.py
│   │   ├── dataset.jsonl    # 78-case evaluation set
│   │   └── case_study/      # Regression case study
│   └── alembic/            # Backend migrations
├── src/
│   ├── app/                # Next.js App Router
│   │   ├── (auth)/         # Login / register
│   │   ├── (chat)/         # Chat interface
│   │   └── api/             # API routes (proxy + auth)
│   ├── components/
│   │   ├── eval/           # Eval list, detail, compare
│   │   ├── query-log/      # Query log explorer
│   │   ├── metrics/        # Metrics dashboard
│   │   └── strategies/     # Strategy selector
│   └── lib/                # Shared utilities, DB client
├── docker-compose.yml      # Main Compose file
├── docker-compose.prod.yml # Production overlay
└── .env.example            # Environment variable template

```

5.4 Modules Description

Module 1: Document Ingestion Module

The document ingestion module is the entry point for all knowledge that the RAG system can draw on. It accepts a file upload through the REST API, determines the document type, extracts the plain text content, segments it into chunks, generates embeddings for each chunk, and stores everything in PostgreSQL.

The extraction step handles three file formats. For plain text files (.txt), the content is read directly. For PDF files (.pdf), the pdfjs-dist library is used on the frontend to extract text before upload, while the backend can also handle raw PDF bytes using a Python PDF library. For DOCX files (.docx), the mammoth library extracts the text content preserving basic paragraph structure.

One subtle challenge in document ingestion is that the quality of chunking has a significant impact on retrieval quality. A chunk that is too short may not have enough context for the embedding model to represent its meaning accurately. A chunk that is too long may cover multiple topics and produce an embedding that represents all of them weakly. The system uses an adaptive chunking strategy described in Section 5.5.

After chunking, each chunk's text is sent to the OpenAI Embeddings API to obtain a 1,536-dimensional dense vector. The chunks are batched (up to 100 per API call) to reduce the number of API requests and associated latency. The resulting (text, embedding, metadata) tuples are bulk-inserted into the chunks table using PostgreSQL's multi-row INSERT, and the IVFFlat index is updated automatically.

A chunk-preview API endpoint allows developers to inspect how a document would be chunked before committing to a full ingestion. This is useful during corpus development to verify that chunk boundaries fall in sensible places.

Module 2: Retrieval Module

The retrieval module is the component most directly responsible for the quality of answers. It receives a query string and a retrieval strategy selection, and returns a ranked list of document chunks to be included in the answer generation context.

Strategy 1: Vector Similarity

This is the baseline strategy. The query is embedded using the same OpenAI embedding model that was used during ingestion, producing a 1,536-dimensional vector. pgvector then performs cosine similarity search using the `<=>` operator against all chunk embeddings:

```
SELECT c.id, c.content, d.source, d.title,
       1 - (c.embedding <=> $1::vector) AS cosine_similarity
FROM chunks c
JOIN documents d ON d.id = c.document_id
ORDER BY c.embedding <=> $1::vector
LIMIT $2;
```

The IVFFlat index makes this approximate (trading a small amount of recall for significant speed gains), but for the corpus sizes involved, the approximation error is negligible.

Strategy 2: Hybrid Search

The hybrid strategy runs vector similarity and BM25 keyword search concurrently using `asyncio.gather()`, then merges the results with Reciprocal Rank Fusion. BM25 scoring is implemented using PostgreSQL's built-in full-text search (`to_tsvector` and `ts_rank`) with an additional inverse document frequency term computed from the chunk count. RRF with `k=60` is used to merge the two ranked lists without requiring score normalisation.

The concurrency here is important: running both searches sequentially would roughly double the retrieval latency. Running them concurrently means the latency is approximately the maximum of the two individual search latencies, which for well-indexed tables is not much more than either alone.

Strategy 3: Reranking

The reranking strategy begins with a vector similarity search retrieving 20 candidates (more than the final `top_k`). It then calls the LLM with a structured prompt asking it to score each candidate on a 0–10 relevance scale:

```

Given the following question and document excerpt, rate how
relevant the excerpt is to answering the question.
Return only a JSON object: {"score": <number between 0 and 10>}

Question: {query}
Document excerpt: {chunk_content}

```

The 20 candidates are scored in parallel using `asyncio.gather()`, and the `top_k` by LLM score are returned. This adds approximately 800–1,200 ms to the retrieval latency (one LLM call per candidate, 20 parallel calls) but can substantially improve the precision of the retrieved set, particularly for questions where the exact phrasing does not match the document vocabulary.

Strategy 4: Multi-Query

The multi-query strategy uses the LLM to generate three reformulations of the original query, targeting different aspects or phrasings. For example, a query about “how does pgvector handle similarity search” might produce reformulations like “pgvector approximate nearest neighbor algorithms”, “vector cosine distance in PostgreSQL”, and “pgvector index types IVFFlat HNSW”. Each reformulation is embedded and searched independently using vector similarity. The results are merged using RRF across all four result sets (original query plus three reformulations), and the `top_k` deduplicated results are returned.

Module 3: Answer Generation Module

The answer generation module takes the query and the list of retrieved chunks as input, constructs a prompt, and streams the response from the OpenAI Chat Completions API back to the client. The prompt is structured as follows:

A system message instructs the model to answer only based on the provided context, to cite sources using the notation [Source: filename], and to say explicitly that the answer is not in the provided documents if it cannot find sufficient evidence. This grounding instruction is important — without it, models tend to blend retrieved context with their parametric knowledge, producing answers that are plausible but not always grounded in the actual documents.

The user message includes the retrieved chunk contents with their source labels, followed by the user’s question. The streaming is implemented using the OpenAI SDK’s `stream=True` parameter, which returns an async iterator over completion chunks. Each chunk is forwarded directly to the HTTP response as a server-sent event, allowing the frontend to update the chat interface token by token without waiting for the complete response.

After streaming completes, the module computes the total latency, the prompt and completion token counts (obtained from the stream’s usage summary), and the cost in USD, and writes these to the query-log row in PostgreSQL along with the OTel trace ID.

Module 4: Evaluation Harness Module

The evaluation harness is a standalone Python script (`eval/run_eval.py`) that can be run from the command line or triggered via the API. It reads the evaluation dataset from `dataset.jsonl`, runs each case through the retrieval module, computes the metrics, and persists the results.

The dataset format is intentionally simple: each line is a JSON object with a `case_id` (a stable string like “case-01”), a question (the natural language query), and an `expected_source` (the file-name of the document that contains the correct answer). This simplicity makes it easy to add new cases, modify existing ones, or use the same dataset format with other retrieval systems for comparison.

For each case, the harness calls the same `retrieve()` function used by the production query pipeline, so the evaluation results reflect actual production behaviour rather than a separate code path. The results for each case are computed and written to `eval_case_results` immediately, so that if the harness is interrupted partway through, the completed cases are preserved and can be inspected.

After all cases are processed, aggregate metrics are computed and written to the `eval_runs` summary row. The harness prints a summary table to `stdout` showing each metric alongside the current value and the value from the baseline run (if a baseline is configured). This makes it easy to see at a glance whether the current run is better or worse than the baseline.

Module 5: CI Regression Gate Module

The CI gate (`eval/compare_eval.py`) is a short, focused Python script with a single responsibility: given two evaluation run IDs, determine whether the candidate run represents an acceptable regression from the baseline and exit with the appropriate code.

The script fetches both runs from the database via the API, computes the delta for each metric, and formats the results as a Markdown table. When running in a GitHub Actions context (detected by the presence of the `GITHUB_OUTPUT` environment variable), it uses the GitHub CLI (`gh`) to post the table as a comment on the pull request.

The gated metrics are `Hit@5` and `MRR`, chosen because they capture different types of failure. `Hit@5` is a recall metric — it measures whether the correct document appears anywhere in the top 5 results. `MRR` is a precision/ranking metric — it measures how highly the correct document is ranked. A change that introduces a highly-ranked but slightly wrong document (like the distractor case study) would hurt `MRR` while leaving `Hit@5` approximately the same. Gating on both metrics catches regressions that would slip through a recall-only gate.

The tolerance values (2 percentage points for `Hit@5`, 0.02 for `MRR`) were chosen empirically by observing the natural run-to-run variance on the 78-case dataset with the same corpus and model. Variance across runs is typically 0.3–0.8 percentage points for `Hit@5` and 0.004–0.008 for `MRR`, so the tolerances are set conservatively enough to absorb this variance without triggering false positives.

Module 6: Observability Module

The observability module instruments the RAG pipeline without adding significant latency or complexity to the request handling code. The key mechanism is a Python `async` context manager (defined in `app/core/tracing.py`) that wraps each pipeline stage:

```
@asynccontextmanager
async def span(name: str, **attributes):
    with tracer.start_as_current_span(name) as current_span:
        for key, value in attributes.items():
            current_span.set_attribute(key, value)
        try:
            yield current_span
        except Exception as exc:
            current_span.record_exception(exc)
            current_span.set_status(StatusCode.ERROR, str(exc))
            raise
```

This context manager can be used anywhere in the pipeline with a simple `async with span("stage.name"): block`. The span is automatically nested under the current active span in the OTEL context, creating the parent-child hierarchy that produces the waterfall chart in Grafana.

The Prometheus metrics are registered as module-level objects — histograms and counters — when the FastAPI application starts. They are updated at the end of each request using the values recorded during processing (latency, token counts, cost). The `/metrics/prometheus` endpoint returns the current values of all metrics in the Prometheus text format, ready for scraping.

Module 7: Query-Log Explorer Module

The query-log explorer is a frontend page that provides a searchable, filterable table of all entries in the queries database table. Each row shows the question, the retrieval strategy used, the total latency, the cost, the timestamp, and the trace ID. Clicking on a row expands it to show the full answer, the list of retrieved source documents with their similarity scores, and a link to the corresponding trace in Grafana Tempo.

The explorer can be filtered by retrieval strategy, by date range, and by whether the query was generated by the evaluation harness (via the `eval_run_id` filter). This last filter is particularly useful: if a CI run flags a regression, the developer can open the query-log explorer filtered by that evaluation run ID and see exactly which questions produced poor retrievals, in what order the documents were ranked, and at what latency.

Module 8: Authentication Module

Authentication in this project uses Auth.js (NextAuth.js v5) configured with two providers: a Credentials provider for email/password login and a custom Guest provider that automatically creates a temporary user account on first visit. The guest session approach was chosen because it allows anyone to try the system without requiring registration, which is important for demonstration purposes.

Passwords for the Credentials provider are hashed using bcrypt with a salt factor of 10 before being stored. Session state is stored as signed JWTs (using the `AUTH_SECRET` as the signing key) in an HTTP-only cookie, which means the session token is never accessible to JavaScript running in the browser.

Chat history is associated with the authenticated user's ID, so each user sees only their own conversations. Guest users' chat history persists across sessions on the same device (as long as the guest session cookie is valid) but is not tied to an email address.

5.5 Algorithms

Algorithm 1: Adaptive Chunking

Chunking strategy matters more than is often appreciated. A 100-token chunk is too short to be self-contained — it may not contain enough context for the embedding to represent the chunk’s topic accurately, and the retrieved text may not provide sufficient context for the LLM to generate a good answer. A 1,500-token chunk covers too much ground — the embedding averages over the entire content and becomes less specific, and the retrieved text includes more noise than signal.

The adaptive chunking algorithm adjusts chunk size and overlap based on the length of the source document:

```
PROCEDURE adaptive_chunk(document_text):
    estimated_tokens = len(document_text) / 4 // rough approximation

    IF estimated_tokens < 400:
        // Very short document: keep as a single chunk
        chunk_size = estimated_tokens
        overlap = 0

    ELSE IF estimated_tokens < 3000:
        // Medium document: 400-token chunks, 40-token overlap
        chunk_size = 400
        overlap = 40

    ELSE IF estimated_tokens < 10000:
        // Long document: 600-token chunks, 80-token overlap
        chunk_size = 600
        overlap = 80

    ELSE:
        // Very long document: 800-token chunks, 100-token overlap
        chunk_size = 800
        overlap = 100

    chunks = []
    start = 0
    WHILE start < len(document_text):
        end = start + chunk_size * 4 // approximate chars from tokens
        chunk_text = document_text[start:end]
        chunks.append(Chunk(text=chunk_text,
                           start_char=start,
                           end_char=min(end, len(document_text))))
        start += (chunk_size - overlap) * 4

    RETURN chunks
```

The overlap between consecutive chunks ensures that a sentence or paragraph that falls near a chunk boundary is fully represented in at least one chunk. Without overlap, a key sentence that straddles a boundary would be split, and neither resulting chunk would capture its full meaning.

Figure 5.1: Adaptive Chunking Decision Tree (see diagram in printed version)

Algorithm 2: Reciprocal Rank Fusion

RRF is used in both the hybrid search strategy (merging vector and BM25 results) and the multi-query strategy (merging results from multiple query reformulations). The algorithm is simple and parameter-free except for the smoothing constant k :

```

PROCEDURE rrf_merge(rankings: list[list[DocumentID]], k=60):
    score_map = {}

    FOR EACH ranking IN rankings:
        FOR rank, doc_id IN enumerate(ranking, start=1):
            IF doc_id NOT IN score_map:
                score_map[doc_id] = 0.0
            score_map[doc_id] += 1.0 / (k + rank)

    sorted_doc_ids = sort(score_map.keys(),
                        key=lambda d: score_map[d],
                        reverse=True)

    RETURN sorted_doc_ids

```

The smoothing constant $k=60$ was used in the original Cormack et al. paper and has since been validated as a robust default across many retrieval tasks. Setting k lower (e.g., $k=10$) gives more extreme weight to top-ranked documents, making the fusion more sensitive to disagreements between systems. Setting k higher (e.g., $k=100$) smooths out the top-rank advantage and gives more weight to documents that appear consistently across all systems. $k=60$ sits in a sweet spot that is robust to most corpus and query characteristics.

The key property of RRF that makes it useful in this context is that it is rank-based: it does not require the scores from different retrieval systems to be on the same scale or to be calibrated against each other. BM25 scores and cosine similarity scores are not comparable numbers, but their relative rankings within each system are meaningful, and RRF exploits only those rankings.

Algorithm 3: Evaluation Harness — Hit@k and MRR Computation

```

PROCEDURE evaluate(dataset, retrieval_fn, top_k=8):
    all_results = []

    FOR EACH case IN dataset:
        // Retrieve top_k chunks for this question
        chunks = retrieval_fn(case.question, top_k)
        retrieved_sources = [chunk.source FOR chunk IN chunks]

        // Compute Hit@k metrics
        hit_results = {}
        FOR k IN [1, 3, 5, 8]:
            hit_results[f"hit_at_{k}"] = 1
            IF case.expected_source IN retrieved_sources[:k]
            ELSE 0

        // Compute reciprocal rank
        reciprocal_rank = 0.0
        FOR rank, source IN enumerate(retrieved_sources, start=1):
            IF source == case.expected_source:
                reciprocal_rank = 1.0 / rank
                BREAK // Only the first correct result counts

        all_results.append({
            "case_id": case.case_id,
            "hit_at_1": hit_results["hit_at_1"],
            "hit_at_3": hit_results["hit_at_3"],
            "hit_at_5": hit_results["hit_at_5"],
            "hit_at_8": hit_results["hit_at_8"],
            "reciprocal_rank": reciprocal_rank,
            "retrieved_sources": retrieved_sources
        })

    // Aggregate
    n = len(all_results)
    aggregate = {
        "hit_at_1": sum(r["hit_at_1"] FOR r IN all_results) / n,
        "hit_at_3": sum(r["hit_at_3"] FOR r IN all_results) / n,
        "hit_at_5": sum(r["hit_at_5"] FOR r IN all_results) / n,
        "hit_at_8": sum(r["hit_at_8"] FOR r IN all_results) / n,
        "mrr": sum(r["reciprocal_rank"] FOR r IN all_results) / n
    }
    RETURN aggregate, all_results

```

A detail worth noting: MRR counts only the first correct result. If a query has multiple potentially correct source documents, only the one that matches `expected_source` exactly is counted. This is appropriate for the evaluation dataset used in this project, where each question has exactly one designated correct source. For evaluation scenarios with multiple acceptable answers, one would need a variant like Mean Average Precision (MAP) or a set-based recall metric.

5.6 Database Tables

Table 5.4: documents

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique document identifier
source	TEXT	UNIQUE, NOT NULL	File path or URL (used as canonical source identifier in evaluations)
title	TEXT	NOT NULL	Human-readable display title
content_type	TEXT	NOT NULL, DEFAULT 'text'	One of: text, pdf, docx
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp of ingestion

The source column is the key design decision here: it is unique across the table and is used as the expected_source value in the evaluation dataset. This means that if a document is re-ingested (for example, after being updated), the new chunks replace the old ones with the same source, and the evaluation results remain interpretable — a hit for “docs/pgvector-intro.txt” always refers to the same document regardless of when it was ingested.

Table 5.5: chunks

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique chunk identifier
document_id	UUID	FK → documents.id, ON DELETE CASCADE	Parent document
content	TEXT	NOT NULL	Raw text content of the chunk
embedding	vector(1536)	NOT NULL	1536-dimensional dense vector from OpenAI
chunk_index	INTEGER	NOT NULL, DEFAULT 0	Position of this chunk within the document
start_char	INTEGER		Starting character offset in the original document text
end_char	INTEGER		Ending character offset in the original document text
metadata	JSONB	DEFAULT '{}'	Arbitrary additional metadata (page number, section heading, etc.)

The IVFFlat index on the embedding column is defined separately from the table:

```
CREATE INDEX chunks_embedding_idx ON chunks
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

Table 5.6: queries

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique query identifier
question	TEXT	NOT NULL	The user's question or evaluation case question
answer	TEXT		The generated answer (may be NULL for incomplete queries)
retrieved_chunk_ids	JSONB	DEFAULT '[]'	Ordered list of chunk IDs returned by the retrieval step
rag_model	TEXT	DEFAULT 'vector-similarity'	Which retrieval strategy was used
latency_ms	INTEGER		Total wall-clock latency from request receipt to final token
prompt_tokens	INTEGER		Number of tokens in the LLM prompt
completion_tokens	INTEGER		Number of tokens in the LLM completion
cost_usd	NUMERIC(10,6)		Measured cost in USD (prompt + completion tokens at current rates)
trace_id	TEXT		OpenTelemetry trace ID for correlation with Grafana Tempo
eval_run_id	UUID	FK → eval_runs.id, NULLABLE	Set when query was generated by the evaluation harness
created_at	TIMESTAMPTZ	DEFAULT NOW()	Query timestamp

Table 5.7: eval_runs

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique run identifier, used as the reference in CI comparisons
dataset	TEXT	NOT NULL	Name or path of the evaluation dataset used
hit_at_1	NUMERIC		Fraction of cases where the correct source was rank 1
hit_at_3	NUMERIC		Fraction of cases where the correct source was in top 3
hit_at_5	NUMERIC		Fraction of cases where the correct source was in top 5 (gated)
hit_at_8	NUMERIC		Fraction of cases where the correct source was in top 8
mrr	NUMERIC		Mean Reciprocal Rank across all cases (gated)
total_cases	INTEGER		Number of cases in the dataset
completed_cases	INTEGER		Number of cases that completed successfully
model	TEXT		Embedding model name (e.g., text-embedding-3-small)
rag_model	TEXT		Retrieval strategy used for this run
created_at	TIMESTAMPTZ	DEFAULT NOW()	Run creation timestamp

Table 5.8: eval_case_results

Column	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique result identifier
run_id	UUID	FK → eval_runs.id, ON DELETE CASCADE	Parent evaluation run
case_id	TEXT	NOT NULL	Stable case identifier from the dataset (e.g., “case-23”)
question	TEXT	NOT NULL	The evaluation question
expected_source	TEXT	NOT NULL	The document source expected to be retrieved
retrieved_sources	JSONB	DEFAULT ‘[]’	Ordered list of actually retrieved source filenames
hit_at_1	BOOLEAN		Whether correct source was retrieved at rank 1
hit_at_3	BOOLEAN		Whether correct source was retrieved in top 3
hit_at_5	BOOLEAN		Whether correct source was retrieved in top 5
hit_at_8	BOOLEAN		Whether correct source was retrieved in top 8
reciprocal_rank	NUMERIC		1/rank of first correct source (0.0 if not retrieved)
created_at	TIMESTAMPTZ	DEFAULT NOW()	Result creation timestamp

The composite index on (run_id, case_id) makes the comparison query efficient:

```
CREATE INDEX eval_case_results_run_case_idx
ON eval_case_results (run_id, case_id);
```

5.7 Important Source Code

5.7.1 Retrieval Strategy Base Class and Vector Similarity Implementation

```
# backend/app/rag/retrieval_strategies.py

from abc import ABC, abstractmethod
from typing import Any
import asyncio
import structlog
from app.core.tracing import span
from app.db.session import get_db_pool
from app.llm.openai_client import get_llm_client
from app.rag.types import RetrievedChunk

logger = structlog.get_logger()

class RetrievalStrategy(ABC):
    @abstractmethod
    async def retrieve(
        self, query: str, top_k: int,
        filters: dict[str, Any] | None = None
    ) -> list[RetrievedChunk]:
        pass

class VectorSimilarityStrategy(RetrievalStrategy):
    async def retrieve(self, query: str, top_k: int,
                      filters=None) -> list[RetrievedChunk]:
        async with span("openai.embedding"):
            embedding = await get_llm_client().embed(query)

        async with span("db.vector_search"):
            pool = await get_db_pool()
            async with pool.acquire() as conn:
                rows = await conn.fetch("""
                    SELECT c.id, c.content, d.source, d.title,
                           1 - (c.embedding <=> $1::vector) AS score
                    FROM chunks c
                    JOIN documents d ON d.id = c.document_id
                    ORDER BY c.embedding <=> $1::vector
                    LIMIT $2
                """, embedding, top_k)
            return [RetrievedChunk(**dict(row)) for row in rows]
```

5.7.2 Hybrid Search Strategy with RRF

```
# backend/app/rag/retrieval_strategies.py (continued)

class HybridSearchStrategy(RetrievalStrategy):
    async def retrieve(self, query: str, top_k: int,
                      filters=None) -> list[RetrievedChunk]:
        async with span("openai.embedding"):
            embedding = await get_llm_client().embed(query)

        # Run both searches concurrently
        vector_results, bm25_results = await asyncio.gather(
            self._vector_search(embedding, top_k * 3),
            self._bm25_search(query, top_k * 3)
        )
        merged = self._rrf_merge(vector_results, bm25_results)
        return merged[:top_k]

    async def _bm25_search(self, query: str,
                          limit: int) -> list[RetrievedChunk]:
        pool = await get_db_pool()
        async with pool.acquire() as conn:
            rows = await conn.fetch("""
                SELECT c.id, c.content, d.source, d.title,
                       ts_rank(
                           to_tsvector('english', c.content),
                           plainto_tsquery('english', $1)
                       ) AS score
                FROM chunks c
                JOIN documents d ON d.id = c.document_id
                WHERE to_tsvector('english', c.content)
                       @@ plainto_tsquery('english', $1)
                ORDER BY score DESC
                LIMIT $2
            """, query, limit)
            return [RetrievedChunk(**dict(row)) for row in rows]

    def _rrf_merge(self, *rankings, k: int = 60) -> list[RetrievedChunk]:
        scores: dict[str, float] = {}
        chunk_map: dict[str, RetrievedChunk] = {}
        for ranking in rankings:
            for rank, chunk in enumerate(ranking, start=1):
                scores[chunk.id] = scores.get(chunk.id, 0) + 1 / (k + rank)
                chunk_map[chunk.id] = chunk
        sorted_ids = sorted(scores, key=scores.__getitem__, reverse=True)
        return [chunk_map[cid] for cid in sorted_ids]
```

5.7.3 Evaluation Harness Core Loop

```
# backend/eval/run_eval.py (key section)

async def evaluate_all_cases(
    dataset: list[dict],
    run_id: str,
    strategy: str = "vector-similarity"
) -> dict:
    retrieval_fn = get_strategy(strategy)
    results = []

    for case in dataset:
        chunks = await retrieval_fn.retrieve(case["question"], top_k=8)
        sources = [c.source for c in chunks]
        expected = case["expected_source"]

        hits = {
            f"hit_at_{k}": int(expected in sources[:k])
            for k in [1, 3, 5, 8]
        }
        rr = next(
            (1.0 / (i + 1) for i, s in enumerate(sources) if s == expected),
            0.0
        )

        result = {"case_id": case["case_id"],
                  "question": case["question"],
                  "expected_source": expected,
                  "retrieved_sources": sources,
                  "reciprocal_rank": rr,
                  **hits}
        results.append(result)
        await persist_case_result(run_id, result)

    n = len(results)
    aggregate = {
        f"hit_at_{k}": sum(r[f"hit_at_{k}"] for r in results) / n
        for k in [1, 3, 5, 8]
    }
    aggregate["mrr"] = sum(r["reciprocal_rank"] for r in results) / n
    await persist_run_summary(run_id, aggregate, n)
    return aggregate
```

5.7.4 CI Regression Gate

```
# backend/eval/compare_eval.py

import sys
from app.db import fetch_eval_run, fetch_eval_summary

TOLERANCES = {"hit_at_5": 0.02, "mrr": 0.02}

def compare(baseline_id: str, candidate_id: str) -> int:
    baseline = fetch_eval_summary(baseline_id)
    candidate = fetch_eval_summary(candidate_id)
    failures = []

    print_header()
    for metric in ["hit_at_1", "hit_at_3", "hit_at_5", "hit_at_8", "mrr"]:
        delta = candidate[metric] - baseline[metric]
        tolerance = TOLERANCES.get(metric)
        gated = tolerance is not None
        verdict = ""
        if gated and delta < -tolerance:
            verdict = "🔴 FAIL"
            failures.append(f"{metric}: {delta:+.4f}")
        elif gated:
            verdict = "🟢 PASS"
        print_row(metric, baseline[metric], candidate[metric], delta, verdict)

    if failures:
        print(f"\nREGRESSION: {' ', ' '.join(failures)}")
        print("Merge blocked.")
        return 1

    print("\nNo regression beyond tolerance. Merge allowed.")
    return 0

if __name__ == "__main__":
    baseline_id = sys.argv[1]
    candidate_id = sys.argv[2]
    sys.exit(compare(baseline_id, candidate_id))
```

5.7.5 OpenTelemetry Span Context Manager

```
# backend/app/core/tracing.py

from contextlib import asynccontextmanager
from opentelemetry import trace
from opentelemetry.trace import StatusCode

tracer = trace.get_tracer(__name__)

@asynccontextmanager
async def span(name: str, **attributes):
    """
    Async context manager for OTel spans.
    Automatically sets error status on exception and re-raises.
    Degrades to a no-op if no tracer provider is configured.
    """
    with tracer.start_as_current_span(name) as current_span:
        for key, value in attributes.items():
            current_span.set_attribute(key, str(value))
        try:
            yield current_span
        except Exception as exc:
            current_span.record_exception(exc)
            current_span.set_status(StatusCode.ERROR, str(exc))
            raise
```

5.7.6 Frontend Chat Streaming Route Handler

```
// src/app/api/chat/route.ts

import { auth } from "@app/(auth)/auth";
import { BACKEND_URL } from "@lib/constants";

export async function POST(request: Request) {
  const session = await auth();
  if (!session?.user) {
    return new Response("Unauthorized", { status: 401 });
  }

  const { messages, ragModel = "vector-similarity" } = await request.json();
  const lastMessage = messages[messages.length - 1];

  const backendResponse = await fetch(
    `${BACKEND_URL}/api/v1/query`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "X-API-Key": process.env.BACKEND_API_KEY ?? "",
      },
      body: JSON.stringify({
        query: lastMessage.content,
        rag_model: ragModel,
      }),
    }
  );

  if (!backendResponse.ok) {
    return new Response("Backend error", {
      status: backendResponse.status,
    });
  }

  // Stream the response body directly - no buffering
  return new Response(backendResponse.body, {
    headers: {
      "Content-Type": "text/event-stream",
      "Cache-Control": "no-cache",
      Connection: "keep-alive",
    },
  });
}
```

5.8 Application Screenshots

The following screenshots illustrate the main views of the application as they appear during a typical session. The descriptions reference the running application at <http://localhost:3000>.

Figure 5.2 — Chat Interface with Streaming Answer and Inline Citations The main chat interface shows a user query in the input box and a streaming answer appearing above it. The answer includes citations formatted as [Source: filename.pdf], and a small observability panel below the answer shows the total latency (3.2 seconds), token count (847 prompt + 312 completion), cost (\$0.00042), and a clickable trace ID link.

Figure 5.3 — Evaluation Run List View The /eval/runs page lists all persisted evaluation runs in reverse chronological order. Each row shows the run ID (truncated UUID), the creation timestamp, the Hit@1, Hit@5, and MRR values, and two action buttons: “View Detail” and “Compare”. Runs can be selected using checkboxes for comparison.

Figure 5.4 — Evaluation Run Comparison View The comparison view shows two runs side by side. The top section shows the per-metric summary table with baseline values, candidate values, deltas, and a pass/fail verdict for each gated metric. Below this, a collapsible per-case table shows all 78 cases, with rows highlighted in red where the candidate run missed a case that the baseline hit, and in green where the candidate hit a case that the baseline missed.

Figure 5.5 — Query Log Explorer The query-log explorer shows a filterable, sortable table of all queries. Each row shows the question (truncated to 60 characters), the retrieval strategy, latency, cost, and timestamp. Expanding a row shows the full question and answer, the ordered list of retrieved source documents, and the trace ID with a link to Grafana.

Figure 5.6 — Metrics Dashboard The metrics page shows a set of time-series charts rendered from Prometheus data via the backend. Charts include p50/p95/p99 latency per route (line charts), request volume (area chart), error rate (bar chart), and cumulative cost (area chart). A pipeline-stage latency chart shows p95 latency separately for the embedding step, the vector search step, and the generation step.

CHAPTER 6

TESTING AND RESULTS

6.1 Test Plan

The testing strategy for this project is organised into four levels, each addressing a different scope of verification.

Unit testing verifies individual functions and classes in isolation. For the backend, this covers the chunking algorithm (correct chunk counts and boundaries for various document lengths), the evaluation metric computation (correct Hit@k and MRR for various rank positions), the RRF merge algorithm (correct score ordering and deduplication), and the CI gate comparison logic (correct exit code for various delta values).

Integration testing verifies that components work together correctly, with real database connectivity. Backend integration tests run against a PostgreSQL instance with pgvector (provided as a Docker service in the CI environment) and verify that API endpoints return correct HTTP codes, that database writes persist correctly and are retrievable, and that streaming endpoints deliver data without errors.

System testing verifies the complete end-to-end behaviour of the application. Playwright browser automation tests drive the chat interface, the evaluation pages, and the query-log explorer through realistic scenarios. These tests include accessibility checks using axe-core, verifying WCAG 2.1 AA compliance.

Evaluation benchmarking is a form of empirical validation specific to this system. It runs all four retrieval strategies against the 78-case evaluation corpus and records the results, verifying that the strategies perform within expected ranges and that the measurement methodology is reproducible.

6.2 Test Cases

Unit Test Cases — Backend

Table 6.1: Unit Test Cases — Backend

TC ID	Test Function	Module	Input	Expected Output	Status
UT-01	test_chunk_short_document	adaptive_chunking	200-token text	1 chunk, no overlap, start_char=0	Pass
UT-02	test_chunk_medium_document	adaptive_chunking	1500-token text	Multiple 400-token chunks with 40-token overlap	Pass
UT-03	test_chunk_long_document	adaptive_chunking	8000-token text	Multiple 800-token chunks, start/end correct	Pass
UT-04	test_chunk_boundary_alignment	adaptive_chunking	Text with paragraph breaks	Chunks do not break mid-sentence	Pass
UT-05	test_chunk_preserves_all_content	adaptive_chunking	Any text	Union of all chunk content covers full text	Pass
UT-06	test_rrf_single_ranking	rrf_merge	One ranking list	Returns list in same order	Pass
UT-07	test_rrf_deduplication	rrf_merge	Two rankings with overlap	Each doc appears once in output	Pass
UT-08	test_rrf_top_scored_doc_first	rrf_merge	Known overlap at rank 1	Overlapping doc scores highest	Pass
UT-09	test_rrf_k_smoothing	rrf_merge	k=60 vs k=10	k=10 gives more extreme top-rank weight	Pass
UT-10	test_hit_at_k_source_at_rank_1	eval harness	expected_source at position 0	hit_at_1=1, hit_at_3=1, hit_at_5=1, rr=1.0	Pass
UT-11	test_hit_at_k_source_at_rank_3	eval harness	expected_source at position 2	hit_at_1=0, hit_at_3=1, hit_at_5=1, rr=0.333	Pass
UT-12	test_hit_at_k_source_at_rank_6	eval harness	expected_source	hit_at_1=0, hit_at_3=0, hit_at_5=0, hit_at_8=1,	Pass

TC ID	Test Function	Module	Input	Expected Output	Status
			source at position 5	rr=0.167	
UT-13	test_hit_at_k_source_not_retrieved	eval harness	expected_source not in results	All hit_at_k=0, rr=0.0	Pass
UT-14	test_mrr_mean_computation	eval harness	Cases with rr=[1.0, 0.5, 0.333]	mrr=0.611	Pass

Table 6.2: Unit Test Cases — Evaluation Module

TC ID	Test Function	Input	Expected Outcome	Status
UT-15	test_compare_no_regression	baseline=0.840 mrr, candidate=0.830	Within tolerance (0.02), exit=0	Pass
UT-16	test_compare_mrr_regression	baseline=0.840 mrr, candidate=0.810	Delta=-0.030 > tolerance, exit=1	Pass
UT-17	test_compare_hit5_regression	baseline=0.949, candidate=0.910	Delta=-0.039 > tolerance, exit=1	Pass
UT-18	test_compare_both_regress	Both MRR and Hit@5 below tolerance	exit=1, both failures reported	Pass
UT-19	test_compare_improvement	candidate better than baseline	exit=0 regardless of improvement size	Pass
UT-20	test_compare_null_run	Invalid run_id passed	Raises ValueError, does not crash silently	Pass

Integration Test Cases**Table 6.3: Integration Test Cases — API Endpoints**

TC ID	Endpoint	Method	Test Scenario	Expected Response	Status
IT-01	/api/v1/health	GET	Service up and DB connected	200 {"status": "ok", "db": "connected"}	Pass
IT-02	/api/v1/ingest	POST	Upload valid TXT file	201 {doc_id, chunk_count > 0}	Pass
IT-03	/api/v1/ingest	POST	Upload valid PDF (multi-page)	201 {doc_id, chunk_count > 1}	Pass
IT-04	/api/v1/ingest	POST	Upload valid DOCX file	201 {doc_id, chunk_count > 0}	Pass
IT-05	/api/v1/ingest	POST	Upload unsupported file type (.xls)	422 Unprocessable Entity	Pass
IT-06	/api/v1/ingest	POST	Re-ingest document with same source	200 with updated chunk count (idempotent)	Pass
IT-07	/api/v1/query	POST	Valid query, vector-similarity strategy	200 SSE stream, contains answer tokens	Pass
IT-08	/api/v1/query	POST	Valid query, hybrid-search strategy	200 SSE stream, contains answer tokens	Pass
IT-09	/api/v1/query	POST	Valid query, reranking strategy	200 SSE stream, longer latency expected	Pass
IT-10	/api/v1/query	POST	Valid query, multi-query strategy	200 SSE stream, contains answer tokens	Pass
IT-11	/api/v1/query	POST	Empty question string	422 Unprocessable Entity	Pass

TC ID	Endpoint	Method	Test Scenario	Expected Response	Status
IT-12	/api/v1/query	POST	Query after ingesting a document	Retrieved sources include ingested document	Pass
IT-13	/api/v1/eval/runs	POST	Create evaluation run	201 {run_id}	Pass
IT-14	/api/v1/eval/runs	GET	List runs (one exists)	200 [{run_id, hit_at_1, mrr, ...}]	Pass
IT-15	/api/v1/eval/runs/{id}	GET	Fetch specific run	200 {run details + case results}	Pass
IT-16	/api/v1/eval/runs/{id}	GET	Fetch non-existent run ID	404 Not Found	Pass
IT-17	/api/v1/eval/compare	GET	Compare two run IDs	200 {deltas per metric, per-case changes}	Pass
IT-18	/api/v1/eval/runs/{id}/export	GET	Export run as JSON	200 JSON document with full run data	Pass
IT-19	/api/v1/eval/runs/{id}/export	GET	Export run as CSV	200 CSV with one row per case	Pass
IT-20	/api/v1/query-log	GET	List query log entries	200 paginated list of query records	Pass
IT-21	/metrics/prometheus	GET	Prometheus scrape	200 Prometheus text format with expected metrics	Pass

System Test Cases

Table 6.4: System Test Cases — End-to-End Scenarios

TC ID	Scenario	Steps	Expected Outcome	Status
ST-01	First-visit guest session	Open app, no login	Guest user created, chat UI accessible	Pass
ST-02	Upload document and query	Upload TXT → Ask question about content	Answer cites the uploaded document	Pass
ST-03	Strategy selection persists	Select reranking, send query	Subsequent queries use reranking until changed	Pass
ST-04	Eval run full cycle	Run CLI harness → Open /eval/runs	Run appears in list with correct metrics	Pass
ST-05	Eval comparison with pass	Two same-corpus runs → Compare	Green PASS for both gated metrics	Pass
ST-06	Eval comparison with regression	Baseline + regressed runs → Compare	Red FAIL for MRR, visual diff of flipped cases	Pass
ST-07	CI gate blocks merge	compare_eval.py with regression runs	Exit code 1, REGRESSION message printed	Pass
ST-08	CI gate passes merge	compare_eval.py with similar runs	Exit code 0, PASS message printed	Pass
ST-09	Query log shows trace ID	Send query → View query log	Trace ID present, link to Grafana (if up)	Pass
ST-10	Filter query log by eval run	Run evaluation → Filter by run ID	Only eval-generated queries shown	Pass
ST-11	Metrics endpoint has latency hist	Send 5 queries → GET /metrics/prometheus	rag_request_duration_seconds histogram present	Pass
ST-12	Accessibility — chat page	Playwright + axe-core on chat view	Zero critical or serious axe violations	Pass
ST-13	Accessibility — eval list	Playwright + axe-core on eval/runs	Zero critical or serious axe violations	Pass

6.3 Unit Testing

Unit tests for the backend are implemented using pytest and pytest-asyncio. The test suite is organised into two packages: tests/unit/ (pure unit tests with no external dependencies, run entirely in memory) and tests/integration/ (tests requiring a live PostgreSQL connection, run in CI with a Postgres service container).

The chunking tests cover a systematic sweep of input lengths — from very short (100 tokens) to very long (20,000 tokens) — verifying that the adaptive algorithm selects the correct chunk size at each length threshold, that the overlap between consecutive chunks matches the configured overlap value, and that the union of all chunk contents covers the complete original text without gaps.

The metric computation tests cover all the boundary cases that are easy to get wrong: what happens when the expected source is the very last result in the top-k list (reciprocal rank should be $1/k$, not 0), what happens when `expected_source` appears at multiple positions (only the first should count), and what happens when the results list is empty (all metrics should be 0 without raising exceptions).

Backend test coverage is enforced in CI using `pytest-cov` with a minimum coverage floor of 70%. Below this floor, the CI build fails and the pull request cannot be merged. This floor was set based on the fact that some code paths (error handling for edge cases in the OpenAI API, database connection failures) are difficult to test without mock infrastructure, but the core pipeline logic must be fully covered.

6.4 Integration Testing

Integration tests run against a real PostgreSQL instance with the `pgvector` extension and the full Alembic migration history applied. The CI environment (GitHub Actions) provides a Postgres 15 service container with `pgvector` pre-installed.

The integration tests cover the API endpoints systematically: each endpoint is tested for the happy path (valid input, expected success response) and for error cases (invalid input, missing required fields, non-existent resource IDs). The streaming endpoint tests verify that the HTTP response uses the correct Content-Type header (`text/event-stream`) and that the response body contains at least some data (the first few characters of the answer) within a reasonable timeout.

Database write verification tests check not only that an API call returns 201 Created, but also that the data was actually persisted correctly by fetching it back through the read endpoint and verifying the values. This is particularly important for the evaluation harness: a test that only checks the HTTP response code would miss a bug where the harness computes metrics correctly but writes them to the wrong database table.

6.5 System Testing

System tests are implemented using Playwright, the cross-browser automation library maintained by Microsoft. The tests run against a full stack environment started by the test harness using Docker Compose before the test suite begins.

The chat interface tests drive the complete user flow: typing a question in the input field, waiting for the streaming response to complete, verifying that the response contains at least one citation in the expected format ([Source: filename]), and verifying that the query-log record was created (by

navigating to the query-log explorer and verifying the question appears in the list).

The evaluation tests drive the frontend pages: navigating to /eval/runs, verifying that a newly created run appears in the list within a reasonable time, clicking the View Detail button and verifying that the per-case table is populated, and clicking the Compare button for two runs and verifying that the comparison table shows the correct deltas.

The accessibility tests use axe-core (integrated with Playwright through @axe-core/playwright) to scan each page for WCAG 2.1 AA violations after page load. The tests assert that zero critical or serious violations are present. This ensures the application is usable with screen readers and keyboard-only navigation.

6.6 Results and Analysis

6.6.1 Retrieval Strategy Benchmark Results

The four retrieval strategies were benchmarked against the 78-case evaluation corpus using text-embedding-3-small for embedding and gpt-4o-mini for generation and reranking. The benchmark was run three times and the results were averaged to account for run-to-run variance.

Table 6.5: Retrieval Strategy Benchmark Results

Strategy	Hit@1	Hit@3	Hit@5	Hit@8	MRR	Latency p50	Latency p95	Cost / 1k queries
Vector Similarity	76.9%	88.5%	94.9%	97.4%	0.840	227 ms	331 ms	\$0.0002
Hybrid Search	75.6%	87.2%	94.9%	97.4%	0.825	220 ms	292 ms	\$0.0002
Reranking	73.1%	88.5%	98.7%	98.7%	0.831	1,119 ms	1,599 ms	\$0.2361
Multi-Query	73.1%	87.2%	94.9%	97.4%	0.827	2,089 ms	2,906 ms	\$0.0372

Corpus: 78-case bundled dataset, text-embedding-3-small + gpt-4o-mini, top_k=8

Several observations from these results deserve detailed discussion.

Vector similarity performs best on MRR. This is somewhat counterintuitive — one might expect a more sophisticated strategy to achieve better ranking quality. The explanation is that on this particular corpus (relatively short, focused documents with clear semantic distinctions between topics), the embedding model captures the topic well enough that the top-1 result is usually correct. The more complex strategies introduce additional computations that occasionally displace the best result.

Reranking achieves the best Hit@5 but at significant cost. The 3.8 percentage-point improvement in Hit@5 (94.9% → 98.7%) comes at the cost of approximately 5× higher latency and roughly 1,000× higher API cost. Whether this trade-off is acceptable depends entirely on the ap-

plication’s latency requirements and budget. For a user-facing chat application where latency matters, the extra 900 ms may not be worth 3.8pp of recall. For an automated batch processing pipeline where latency is irrelevant, it might be worth it.

Multi-query underperforms on this corpus. This is the most practically important finding because it contradicts the intuition that “more queries = more retrieval coverage”. On this corpus, the query reformulations generated by the LLM do not produce documents that were missed by the original query — they retrieve largely the same documents. The increased latency (2× over vector similarity) and cost (80× over vector similarity) are entirely waste. This finding illustrates precisely why benchmarking is necessary: reasonable-sounding improvements must be measured, not assumed.

Hybrid search provides marginal improvement in latency over vector similarity. This is counterintuitive (one would expect two searches to be slower than one), but it reflects the fact that BM25 search using PostgreSQL full-text search is extremely fast (under 10 ms in most cases), while vector similarity search dominates the retrieval latency. Running them concurrently adds almost no overhead.

6.6.2 Regression Case Study Results

The regression case study reproduces a scenario that is unfortunately common in production RAG systems: adding seemingly helpful documents that inadvertently degrade retrieval quality by competing with more specific sources.

Setup: Four broad overview documents (a vector-index cheatsheet, a retrieval-methods glossary, an evaluation-metrics glossary, and an embeddings overview) were added to the corpus alongside the existing 27 focused documents. These documents mention every keyword relevant to the evaluation questions but provide only shallow coverage of each topic.

Results:

Table 6.6: Regression Case Study — Metric Comparison

Metric	Baseline (27 docs)	Regressed (31 docs)	Delta	Gate Status
Hit@1	76.9%	70.5%	-6.4 pp	Warning (not gated)
Hit@3	88.5%	91.0%	+2.6 pp	Pass
Hit@5	94.9%	97.4%	+2.6 pp	Pass (gated)
Hit@8	97.4%	98.7%	+1.3 pp	Pass
MRR	0.840	0.812	-0.028	🔴 FAIL (gated)

CI Gate Result: **exit code 1 — merge blocked**

The case study result validates the central design decision to gate on both MRR and Hit@5 rather than on Hit@5 alone. Hit@5 *improved* when the distractor documents were added (+2.6pp), because the broad overview documents push some peripherally relevant documents from rank 6–7

into the top 5, improving top-5 recall. A Hit@5-only gate would have passed this change and allowed the degraded corpus to be deployed.

MRR, however, correctly identifies the regression: the average rank of the correct source dropped, because the distractor documents push the authoritative source from rank 1 to rank 2 or 3 for 12 of the 78 questions. The number of questions answered by the correct document at rank 1 fell from 60 to 55. This is a meaningful quality degradation that the gate correctly catches.

The detailed per-case comparison in the eval comparison UI shows exactly which 12 questions were affected and which distractor document outranked the correct source for each one. This pinpoints exactly what to investigate and fix — in this case, the fix would be to either remove the distractor documents or to apply metadata filtering that distinguishes general overview documents from authoritative technical sources.

6.6.3 Observability Validation

To validate the OpenTelemetry instrumentation, a series of 20 queries were sent to the running system and the resulting traces were inspected in Grafana Tempo. All 20 traces showed the expected span hierarchy: a root span (`rag.request`) with two child spans (`rag.retrieve` and `rag.generate`), each containing their own nested spans (`openai.embedding` and `db.vector_search` under `retrieve`; `openai.chat` under `generate`).

The span attributes were verified: model name, token counts, and chunk count were correctly recorded on each span. The `trace_id` on each span matched the `trace_id` stored in the corresponding queries table row, confirming the three-way correlation between traces, the query-log database, and the structured log output.

Prometheus metrics were verified by examining the `/metrics/prometheus` endpoint after running the 20 queries. The `rag_request_duration_seconds` histogram showed the correct number of observations (20) and the sum of observed durations was consistent with the total latency recorded in the query-log table.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project set out to address a practical, consequential gap in the way RAG systems are built and operated. The starting observation — that it is easy to build a RAG demo and very hard to know whether it is actually working well — is not a niche concern. It affects every organisation that has deployed or is considering deploying a document question-answering system, and it is not addressed by any existing tool in a satisfactory way.

The system described in this report addresses the gap through six specific mechanisms: a persisted evaluation harness that runs the same dataset on every pipeline change, a run comparison view keyed by stable case identifiers that pinpoints exactly which questions regressed, a CI gate that blocks merges when retrieval quality drops, per-stage OpenTelemetry tracing that attributes latency to specific pipeline components, a query-log explorer that correlates CI failures with production traces, and a shared PostgreSQL database that provides a common identifier across all of these.

The regression case study demonstrates the system’s flagship capability working as intended. Adding four broad overview documents to the corpus degraded MRR from 0.840 to 0.812 — a regression that would have been invisible to Hit@5-only monitoring, would likely not have been caught in code review, and would have silently shipped to production. The CI gate caught it, blocked the merge, and the per-case comparison identified the specific 12 questions affected and the specific distractor documents responsible.

The retrieval strategy benchmark produced findings that were genuinely informative rather than confirming prior assumptions. Vector similarity — the simplest and cheapest strategy — achieved the best MRR on the test corpus. Reranking improved Hit@5 by 3.8 percentage points at approximately 1,000× higher cost. Multi-query, despite its theoretical appeal, did not improve retrieval on this corpus while adding significant latency and cost. These are not results that could be derived from first principles — they had to be measured, and the evaluation infrastructure makes that measurement routine.

The observability layer provided immediate practical value during development. On multiple occasions, tracing the span waterfall revealed that what appeared to be “slow generation” was actually slow embedding (because the embedding API had higher latency at a particular time of day) or slow vector search (because the IVFFlat index needed tuning after a batch ingestion). Without per-stage spans, all of these latency attributions would have required manual instrumentation and logging changes. With the OTel instrumentation in place, the answer was available in Grafana within seconds of sending the slow query.

The system is fully functional, containerised, tested, and deployable. The CI pipeline runs linting, type checking, unit tests (with 70%+ coverage enforcement), integration tests against a real PostgreSQL instance, and Playwright end-to-end tests including accessibility checks on every pull

request. The Docker Compose deployment brings up the complete stack in a single command. Cloud deployment guides for Render and Azure Container Apps allow the system to be deployed to a production environment with minimal additional configuration.

What this project has built is not merely a demonstration of RAG capabilities — those are already abundant. It is an engineering argument that RAG quality regression deserves to be treated as seriously as code correctness, and it provides the infrastructure to back that argument up with practice. A team using this system is positioned to answer the question “did my last change make retrieval better or worse?” with the same confidence and speed that they can answer “did my last change break the unit tests?”

7.2 Limitations

Limitation 1: Corpus and dataset scale. The evaluation dataset contains 78 cases against a 27-document corpus. This is a deliberate choice for a demonstration system — small enough to run quickly and cheaply in CI, large enough to produce statistically meaningful metric comparisons. However, production RAG deployments typically operate at scales of thousands of documents and hundreds to thousands of evaluation cases. The evaluation harness would need to be parallelised and possibly distributed across multiple workers to handle these scales in a reasonable CI window.

Limitation 2: Single-tenant architecture. The current system does not implement per-user data isolation at the database level. All authenticated users share the same document corpus and can query all ingested documents. For a multi-tenant deployment — where different organisations’ documents should not be visible to each other — row-level security (RLS) in PostgreSQL or a schema-per-tenant approach would be required. This is a known limitation and is documented in the project’s HARDENING.md file.

Limitation 3: Retrieval-only evaluation metrics. The evaluation harness measures retrieval quality (whether the correct document was retrieved) but not answer quality (whether the generated answer was correct and faithful to the retrieved context). A system could have high Hit@5 while generating incorrect or misleading answers if the generation component has problems. Addressing this would require an LLM-as-judge evaluation step (similar to RAGAS) that assesses faithfulness and answer correctness, which adds cost and complexity to the evaluation pipeline.

Limitation 4: Embedding model lock-in. The system is designed around OpenAI’s text-embedding-3-small model. All stored chunk embeddings are 1,536-dimensional vectors in the embedding space of this specific model version. Switching to a different embedding model (whether a newer OpenAI model or an open-source alternative) would require re-embedding and re-indexing all stored chunks, since embeddings from different models are not comparable. This is a fundamental property of dense retrieval systems, not a design flaw specific to this implementation, but it is a constraint that operators should be aware of.

Limitation 5: No automatic baseline promotion. The CI baseline is pinned manually — a developer must deliberately update the baseline run ID after a successful improvement. An automated system that promoted the baseline after every passing merge would reduce this manual step but

would also risk allowing gradual drift (many small acceptable changes accumulating into a significant quality difference from the original baseline). The current manual approach gives more control but requires discipline.

Limitation 6: Limited authentication options. The current implementation supports only guest sessions and email/password authentication. Production deployments typically require integration with organisational identity providers (SAML, OIDC, Google/Microsoft OAuth). Adding SSO support would require additional Auth.js provider configuration.

7.3 Future Enhancements

Enhancement 1: LLM-as-Judge Generation Quality Metrics

The most impactful addition to the evaluation system would be extending the harness to include generation quality metrics alongside the current retrieval metrics. Using an LLM to score each answer for faithfulness (does the answer contain claims that contradict the retrieved context?) and answer relevance (does the answer actually address the question?), the system could gate on end-to-end quality rather than retrieval quality alone. The cost of this enhancement can be controlled by sampling — running LLM judging on a random 20% of evaluation cases rather than all 78 — which would add approximately \$0.15 per CI run for the reranking strategy.

Enhancement 2: Automatic Baseline Promotion

A CI workflow step that automatically promotes the current baseline when a pull request is merged to the main branch would eliminate the manual baseline update step. The promotion would copy the `run_id` from the merge commit's CI artifacts to a pinned configuration file in the repository. This would need a guard against promoting a run that only barely passed the tolerance — for example, requiring that the new run's metrics are at least as good as the current baseline (not just within tolerance of it) before promoting.

Enhancement 3: Local Embedding Model Support

Integrating with Ollama or the Hugging Face Transformers library to support local embedding models would remove the dependency on the OpenAI API for the embedding step. Models like `nomic-embed-text` or `all-MiniLM-L6-v2` run on CPU and produce good-quality embeddings for many domains. This would enable air-gapped deployment (for organisations with strict data residency requirements), eliminate embedding API costs, and allow experimentation with domain-adapted embedding models without API access.

Enhancement 4: Advanced Retrieval Strategies

Several retrieval strategies from the recent literature have not yet been implemented in this system. RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) builds a hierarchical tree of summaries at multiple levels of abstraction, enabling retrieval at both the detailed and high-level conceptual levels — useful for corpora with many inter-related documents. Self-RAG (Selective Retrieval-Augmented Generation) allows the LLM to decide adaptively when re-

trieval is needed and how many retrieval steps to perform, potentially improving both quality and efficiency. ColBERT-style late-interaction models provide retrieval quality close to cross-encoders at a cost much closer to bi-encoders.

Enhancement 5: Real-Time Quality Monitoring

The current evaluation is batch-only: it runs a fixed dataset through the retrieval pipeline and measures quality at that point in time. Real-time quality monitoring would sample a percentage of live production queries, automatically assess them (using an LLM judge or other automated quality signal), and surface quality trends on the metrics dashboard. This would complement the offline evaluation by providing signal on the actual distribution of production queries rather than a curated evaluation dataset.

Enhancement 6: Grafana Alerting

The system exports Prometheus histograms but does not currently configure any alerting rules. Adding Prometheus alerting rules — for example, alerting when p95 latency exceeds 8 seconds for more than 5 minutes, or when the error rate exceeds 2% — would transform the metrics dashboard from a diagnostic tool into a proactive operational signal. The Grafana Alerting feature allows alerts to be configured in the Grafana UI and sent to Slack, PagerDuty, or email.

Enhancement 7: Document Change Detection and Partial Re-Indexing

When a source document is updated, the current system requires a full re-ingestion: the old document is deleted and all its chunks re-created. For large documents in large corpora, this is wasteful — most of the document may be unchanged. A diff-based re-indexing system that identifies which sections of a document have changed and updates only those chunks would significantly reduce re-ingestion cost and time. This would require storing a content hash per chunk and comparing incoming document sections against stored hashes.

Enhancement 8: Multi-Tenant Support

Implementing per-tenant data isolation using PostgreSQL’s Row Level Security (RLS) feature would enable the system to serve multiple organisations from a single deployment. Each tenant would have their own document corpus, evaluation datasets, and query logs, isolated at the database level from all other tenants. This would require adding a `tenant_id` column to all relevant tables, enabling RLS with appropriate policies, and managing tenant authentication separately from user authentication.

References / Bibliography

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

- [2] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Guo, Q., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*.
- [3] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. T. (2020). Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781.
- [4] Luan, Y., Eisenstein, J., Toutanova, K., & Collins, M. (2021). Sparse, Dense, and Attentional Representations for Text Retrieval. *Transactions of the Association for Computational Linguistics*, 9, 329–345.
- [5] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd Annual International ACM SIGIR Conference*, pp. 758–759.
- [6] Nogueira, R., & Cho, K. (2019). Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.
- [7] Mao, Y., He, P., Liu, X., Shen, Y., Gao, J., Han, J., & Chen, W. (2021). Generation-Augmented Retrieval for Open-Domain Question Answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL-IJCNLP)*, pp. 4089–4100.
- [8] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217*.
- [9] Voorhees, E. M., & Harman, D. K. (Eds.). (2005). *TREC: Experiment and Evaluation in Information Retrieval*. MIT Press.
- [10] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, 28.
- [11] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- [12] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT 2019*, pp. 4171–4186.
- [13] Johnson, J., Douze, M., & Jégou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- [14] Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.

- [15] OpenTelemetry Community. (2024). *OpenTelemetry: An Observability Framework for Cloud-Native Software*. Cloud Native Computing Foundation. Retrieved from <https://opentelemetry.io/>
- [16] Prometheus Authors. (2024). *Prometheus — From Metrics to Insight*. Retrieved from <https://prometheus.io/>
- [17] Tiangolo, S. (2024). *FastAPI: Modern, Fast Web Framework for Building APIs with Python 3.8+*. Retrieved from <https://fastapi.tiangolo.com/>
- [18] Vercel Inc. (2024). *Next.js 15 Documentation*. Retrieved from <https://nextjs.org/docs>
- [19] pgvector Contributors. (2024). *pgvector: Open-source Vector Similarity Search for PostgreSQL*. Retrieved from <https://github.com/pgvector/pgvector>
- [20] OpenAI. (2024). *API Reference — Embeddings and Chat Completions*. Retrieved from <https://platform.openai.com/docs/api-reference>
-
-

Appendix A — Source Code (Key Modules)

The complete source code for this project is maintained at: <https://github.com/revanthchary04/rag-eval-observe>

The following listings include the key modules referenced throughout this report.

A.1 Retrieval Strategy Dispatcher (retrieve.py)

```
# backend/app/rag/retrieve.py

from app.rag.retrieval_strategies import (
    VectorSimilarityStrategy,
    HybridSearchStrategy,
    RerankingStrategy,
    MultiQueryStrategy,
    RetrievalStrategy,
)

_STRATEGIES: dict[str, type[RetrievalStrategy]] = {
    "vector-similarity": VectorSimilarityStrategy,
    "hybrid-search": HybridSearchStrategy,
    "reranking": RerankingStrategy,
    "multi-query": MultiQueryStrategy,
}

def get_strategy(name: str) -> RetrievalStrategy:
    cls = _STRATEGIES.get(name)
    if cls is None:
        valid = list(_STRATEGIES.keys())
        raise ValueError(
            f"Unknown retrieval strategy '{name}'. "
            f"Valid options: {valid}"
        )
    return cls()
```

A.2 Main FastAPI Application (main.py)

```
# backend/app/main.py

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import settings
from app.core.tracing import configure_tracing
from app.core.metrics import configure_metrics
from app.api.routes import ingest, query, eval_runs, query_log, health

configure_tracing()
app = FastAPI(title="RAG Eval API", version="0.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.CORS_ORIGINS,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(health.router, prefix="/api/v1")
app.include_router(ingest.router, prefix="/api/v1")
app.include_router(query.router, prefix="/api/v1")
app.include_router(eval_runs.router, prefix="/api/v1")
app.include_router(query_log.router, prefix="/api/v1")

@app.get("/metrics/prometheus")
async def prometheus_metrics():
    from prometheus_client import generate_latest, CONTENT_TYPE_LATEST
    from fastapi.responses import Response
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)
```

A.3 Strategy Benchmark Script (benchmark_strategies.py)

```
# backend/eval/benchmark_strategies.py

import asyncio
import json
from pathlib import Path
from app.rag.retrieve import get_strategy
from app.db.session import init_db_pool

STRATEGIES = ["vector-similarity", "hybrid-search", "reranking", "multi-query"]
DATASET_PATH = Path(__file__).parent / "dataset.jsonl"

async def benchmark(strategy_name: str, max_cases: int | None = None):
    dataset = [
        json.loads(line)
        for line in DATASET_PATH.read_text().splitlines()
    ]
    if max_cases:
        dataset = dataset[:max_cases]

    strategy = get_strategy(strategy_name)
    hits = {1: 0, 3: 0, 5: 0, 8: 0}
    rr_total = 0.0

    for case in dataset:
        chunks = await strategy.retrieve(case["question"], top_k=8)
        sources = [c.source for c in chunks]
        expected = case["expected_source"]
        for k in [1, 3, 5, 8]:
            if expected in sources[:k]:
                hits[k] += 1
        rr_total += next(
            (1.0 / (i + 1) for i, s in enumerate(sources) if s == expected),
            0.0
        )

    n = len(dataset)
    return {
        "strategy": strategy_name,
        "hit_at_1": hits[1] / n,
        "hit_at_5": hits[5] / n,
        "mrr": rr_total / n,
        "n": n,
    }

async def main():
    await init_db_pool()
    for name in STRATEGIES:
        result = await benchmark(name)
        print(f"{name}: Hit@1={result['hit_at_1']:.1%} "
              f"Hit@5={result['hit_at_5']:.1%} "
              f"MRR={result['mrr']:.3f}")

if __name__ == "__main__":
    asyncio.run(main())
```

Appendix B — SQL Scripts

B.1 Enable pgvector Extension

```
-- Run once per database, requires superuser privileges
CREATE EXTENSION IF NOT EXISTS vector;
```

B.2 Create RAG Core Tables

```
-- documents: source documents ingested into the knowledge base
CREATE TABLE IF NOT EXISTS documents (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  source      TEXT          UNIQUE NOT NULL,
  title       TEXT          NOT NULL,
  content_type TEXT        NOT NULL DEFAULT 'text',
  created_at  TIMESTAMPTZ   DEFAULT NOW()
);

-- chunks: segmented document content with vector embeddings
CREATE TABLE IF NOT EXISTS chunks (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID          NOT NULL
                        REFERENCES documents(id) ON DELETE CASCADE,
  content     TEXT          NOT NULL,
  embedding   vector(1536),
  chunk_index INTEGER       NOT NULL DEFAULT 0,
  start_char  INTEGER,
  end_char    INTEGER,
  metadata    JSONB        DEFAULT '{} '
);

-- Approximate nearest-neighbour index for cosine similarity search
CREATE INDEX IF NOT EXISTS chunks_embedding_idx
  ON chunks USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);
```

B.3 Create Query Audit Log Table

```
-- queries: every query processed by the pipeline (user and eval)
CREATE TABLE IF NOT EXISTS queries (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  question    TEXT          NOT NULL,
  answer      TEXT,
  retrieved_chunk_ids JSONB  DEFAULT '[]',
  rag_model    TEXT          DEFAULT 'vector-similarity',
  latency_ms  INTEGER,
  prompt_tokens INTEGER,
  completion_tokens INTEGER,
  cost_usd    NUMERIC(10, 6),
  trace_id    TEXT,
  eval_run_id UUID          REFERENCES eval_runs(id),
  created_at  TIMESTAMPTZ   DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS queries_eval_run_idx
  ON queries(eval_run_id)
  WHERE eval_run_id IS NOT NULL;

CREATE INDEX IF NOT EXISTS queries_created_at_idx
  ON queries(created_at DESC);
```

B.4 Create Evaluation Tables

```
-- eval_runs: summary of each evaluation harness run
CREATE TABLE IF NOT EXISTS eval_runs (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  dataset     TEXT          NOT NULL,
  hit_at_1    NUMERIC,
  hit_at_3    NUMERIC,
  hit_at_5    NUMERIC,
  hit_at_8    NUMERIC,
  mrr         NUMERIC,
  total_cases INTEGER,
  completed_cases INTEGER,
  model       TEXT,
  rag_model   TEXT,
  created_at  TIMESTAMPTZ  DEFAULT NOW()
);

-- eval_case_results: per-case results for each evaluation run
CREATE TABLE IF NOT EXISTS eval_case_results (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id      UUID          NOT NULL
                        REFERENCES eval_runs(id) ON DELETE CASCADE,
  case_id     TEXT          NOT NULL,
  question    TEXT          NOT NULL,
  expected_source TEXT      NOT NULL,
  retrieved_sources JSONB    DEFAULT '[]',
  hit_at_1    BOOLEAN,
  hit_at_3    BOOLEAN,
  hit_at_5    BOOLEAN,
  hit_at_8    BOOLEAN,
  reciprocal_rank NUMERIC,
  created_at  TIMESTAMPTZ  DEFAULT NOW()
);

-- Composite index for the comparison query (JOIN on run_id, match on case_id)
CREATE INDEX IF NOT EXISTS eval_case_results_run_case_idx
  ON eval_case_results(run_id, case_id);
```

Appendix C — User Manual

C.1 System Requirements

Before starting, ensure the following software is installed on your machine:

- **Docker Desktop** (version 24 or later) — includes both Docker and Docker Compose
- **A web browser** — Chrome, Firefox, Edge, or Safari (latest version)
- **An OpenAI API key** — obtained from <https://platform.openai.com/api-keys>

No other software needs to be installed for basic usage. For development (modifying the code), Python 3.11 and Node.js 20 are additionally required.

C.2 Installation and First Launch

Step 1: Download or clone the source code:

```
git clone https://github.com/revanthchary04/rag-eval-observe.git
cd rag-eval-observe
```

Step 2: Create the configuration file:

```
cp .env.example .env
```

Open the `.env` file in a text editor. Set the following two required values: - `OPENAI_API_KEY` — Your OpenAI API key, beginning with “sk-” - `AUTH_SECRET` — A random 32-character string. Generate one by running: `openssl rand -base64 32` in your terminal and pasting the output.

Step 3: Start the application:

```
docker compose --profile full up -d
```

This command may take several minutes the first time as Docker downloads the required images. Subsequent starts are much faster.

Step 4: Verify that the system is running:

```
curl http://localhost:8000/api/v1/health
```

You should see: `{"status": "ok", "db": "connected"}`

Step 5: Open the application in your browser: `http://localhost:3000`

C.3 Using the Chat Interface

On first visit, a guest session is created automatically — no login is required. You will see the chat interface with an example question shown in the input box.

To ask a question: 1. Type your question in the text input at the bottom of the chat area. 2. Press Enter or click the Send button. 3. The answer will appear above your question, streaming in real time. 4. After the answer completes, you will see inline citations [Source: filename] and an observability panel showing latency, tokens used, and cost.

To upload your own documents: 1. Click the Upload button (paperclip icon) next to the text input. 2. Select one or more files in TXT, PDF, or DOCX format. 3. Wait for the “Document ingested successfully” confirmation message. 4. You can now ask questions about the content of the uploaded documents.

To change the retrieval strategy: 1. Click the Strategy selector dropdown above the chat input. 2. Select one of: Vector Similarity, Hybrid Search, Reranking, Multi-Query. 3. Subsequent queries will use the selected strategy until you change it again.

C.4 Running an Evaluation

The evaluation harness measures how well the retrieval pipeline performs on a fixed set of test questions.

From the command line:

```
cd backend
uv run python eval/run_eval.py
```

Progress is printed to the terminal as each case completes. When all 78 cases are done, a summary table shows Hit@1, Hit@3, Hit@5, Hit@8, and MRR.

From the web interface: Navigate to <http://localhost:3000/eval/runs> to see a list of all completed evaluation runs. Click “View Detail” on any run to see the per-case results. Select two runs using the checkboxes and click “Compare” to see a side-by-side comparison.

C.5 Running the CI Regression Gate

```
cd backend
# Replace <baseline_id> and <candidate_id> with actual run UUIDs
# (visible in the eval runs list in the web interface)
uv run python eval/compare_eval.py <baseline_id> <candidate_id>
```

A table showing the metric comparison is printed to the terminal. The script exits with code 0 if no gated metric has regressed beyond tolerance, or code 1 if regression was detected. In a CI environment, a non-zero exit code will cause the pipeline step to be marked as failed.

C.6 Starting the Observability Stack

```
docker compose --profile observability up -d
```

Grafana will be available at <http://localhost:3001>. The default login is: - Username: `admin` - Password: `admin`

The RAG pipeline dashboard and the trace explorer are pre-configured as datasources and dashboards. No manual configuration is required.

C.7 Stopping the Application

```
docker compose --profile full down
```

This stops all containers but preserves the database data. To also remove the stored data:

```
docker compose --profile full down -v
```

Warning: The `-v` flag deletes all database data including ingested documents, query logs, and evaluation results. Use this only if you want a completely fresh start.

Appendix D — Output Screenshots

The following descriptions correspond to screenshots taken from the running application. In the printed report, these should be replaced by the actual screenshots.

D.1 — Chat Interface (Initial State) The initial state of the chat interface shows the application header with the project name, a strategy selector showing “Vector Similarity” selected, and an empty chat area with an example query hint. The text input field at the bottom has placeholder text “Ask a question about your documents”.

D.2 — Document Upload Dialog After clicking the upload button, a file picker dialog opens showing the allowed file types. After selecting a file, a progress indicator shows “Processing document...” followed by a green success toast notification showing the document title and the number of chunks created.

D.3 — Streaming Answer with Citations A user question “What is the difference between IVFFlat and HNSW indexes in pgvector?” has been sent. The answer is still streaming — approximately half the text is visible, updating in real time. The visible text includes one inline citation “[Source: pgvector-docs.txt]”.

D.4 — Completed Answer with Observability Panel The same question as D.3, but now fully answered. Below the answer is the observability panel showing: Latency: 3.1s | Tokens: 612 prompt / 287 completion | Cost: \$0.00038 | Trace: a7f3c9... (clickable link).

D.5 — Evaluation Runs List The /eval/runs page shows a table with three evaluation runs. Each row shows a truncated run ID, the timestamp, Hit@1 (76.9%), Hit@5 (94.9%), and MRR (0.840) for the baseline run, and slightly different values for two other runs. Two checkboxes are selected for comparison.

D.6 — Evaluation Run Comparison View The comparison view shows the selected runs labeled “Baseline” and “Candidate”. A metrics table shows each metric with its baseline value, candidate value, delta, and a verdict emoji (🟢 for pass, 🔴 for fail). Below, a per-case accordion table shows the 78 cases with three rows highlighted in red (cases where the candidate missed a previously-hit source).

D.7 — Query Log Explorer The query log shows 12 entries (from the most recent evaluation run, filtered by eval_run_id). Each row shows the question text (truncated), the strategy (vector-similarity), latency (range 280–520 ms), cost (\$0.00021–\$0.00038), and a trace ID. One row is expanded showing the full retrieved sources list.

D.8 — System Metrics Dashboard A Grafana dashboard with four panels: (top left) p50/p95/p99 request latency as three time-series lines over the past hour; (top right) total request count as an area chart; (bottom left) per-stage latency breakdown (embed, vector search, generate) as grouped lines; (bottom right) cumulative cost in USD as a rising area chart.

D.9 — CI Gate Terminal Output (Regression Detected) A terminal window showing the output of `python eval/compare_eval.py <baseline_id> <candidate_id>`. A formatted table shows metric comparisons with the MRR row highlighted. Below the table: “🔴 REGRESSION

DETECTED: mrr: -0.0280 (beyond -0.0200 tolerance)” and “Merge blocked.” The terminal shows exit code 1.

D.10 — Grafana Tempo Trace Waterfall A Grafana Tempo trace explorer showing a single request trace. The root span “rag.request” (3,142 ms total) contains two child spans: “rag.retrieve” (512 ms) and “rag.generate” (2,630 ms). “rag.retrieve” contains “openai.embedding” (487 ms) and “db.vector_search” (25 ms). “rag.generate” contains “openai.chat” (2,625 ms). Each span shows its attributes in a side panel.

End of Document

Formatting Note (for Word/PDF conversion): This document is written in Markdown. When converting to Word (.docx) for final submission, apply the following formatting: - Body text: Times New Roman 12pt, 1.5 line spacing, justified alignment - Chapter headings: Times New Roman 16pt bold - Section headings (1.1, 2.1, etc.): Times New Roman 14pt bold - Sub-section headings (1.1.1 etc.): Times New Roman 12pt bold - Margins: Left 1.5 inches, Right 1 inch, Top 1 inch, Bottom 1 inch - Page numbers: Bottom centre, starting from Chapter 1 - Code blocks: Courier New 10pt, single spacing, light grey background - Tables: Times New Roman 11pt, 1.15 line spacing, centred heading row